



BKP: An R Package for Beta Kernel Process Modeling

Jiangyan Zhao

East China Normal University

Kunhai Qing

East China Normal University

Jin Xu

East China Normal University

Abstract

Estimating input-dependent probability surfaces from binary, binomial, categorical, or multinomial response data is a common task in statistics and machine learning. Latent Gaussian process classifiers provide flexible nonparametric models for such problems, but posterior inference with discrete responses typically requires approximation or simulation. We discuss an implementation of probability-scale beta and Dirichlet kernel models in the **BKP** package for R. The package implements the Beta Kernel Process (BKP), which uses kernel-weighted pseudo-count aggregation and beta-binomial conjugacy to obtain closed-form conjugate posterior summaries and posterior predictive distributions for binomial probabilities. It also implements the Dirichlet Kernel Process (DKP) for multi-class responses, together with TwinBKP and TwinDKP, scalable twinning-based global-local approximations for larger datasets. The resulting workflow supports transparent kernel-weighted evidence borrowing, several kernel families, fixed and data-adaptive priors, effective-sample-size calibration, loss-based hyperparameter tuning, and standard S3 methods for fitting, prediction, simulation, visualization, and extraction of posterior summaries. Reproducible examples demonstrate probability-surface estimation, binary and multi-class classification, computational comparison, and real-data applications to *Loa loa* infection prevalence mapping and Mourning Warbler distribution modeling.

Keywords: Beta kernel process, Dirichlet kernel process, conjugate updating, kernel smoothing, scalable computation, kernel-weighted evidence borrowing, R.

1. Introduction

Estimating an input-dependent probability function from binary or binomial observations is a fundamental task in modern statistics and machine learning (Hastie *et al.* 2009; Rolland *et al.* 2019; Murphy 2022). Such problems arise when the goal is to infer an unknown probability surface from individual Bernoulli outcomes or aggregated binomial counts over a continuous input space. Representative applications include binary classification (MacKenzie *et al.* 2014; Wen *et al.* 2025), probability calibration (Sung *et al.* 2020; Dimitriadis *et al.* 2022), relative abundance modeling (Martin *et al.* 2020), longitudinal analysis of patient-reported outcomes (Najera-Zuloaga *et al.* 2018, 2019), infectious disease risk mapping (Diggle *et al.* 2007; Diggle and Giorgi 2016), and adaptive clinical trial design for dose exploration and dose optimization (Benest *et al.* 2022; Zhao *et al.* 2026b).

Classical approaches such as logistic regression and generalized additive models provide computationally efficient tools for binomial regression, but they usually impose either a fixed parametric form or an additive smoothing structure, which may limit their ability to capture complex nonlinear probability surfaces (Hastie *et al.* 2009). Latent Gaussian process (GP) classifiers provide a flexible nonparametric alternative for probabilistic classification (Rasmussen and Williams 2006). However, binary and categorical responses lead to non-Gaussian likelihoods, so posterior inference typically requires approximation or simulation, such as Laplace approximation, expectation propagation, variational inference, or Markov chain Monte Carlo methods (Nickisch and Rasmussen 2008). These approximations can increase computational cost and implementation complexity, motivating probability-scale methods that avoid latent-variable inference and retain closed-form conjugate posterior summaries.

An alternative route is to model the probability surface directly on the probability scale. Rather than placing a GP prior on a latent real-valued function and mapping it to probabilities through a link function, beta-kernel methods borrow information across nearby inputs through kernel weights and produce beta or Dirichlet conjugate updates by aggregating fractional pseudo-counts. Related ideas have appeared under different names in several research communities, including smoothed beta-distribution models for robotic grasping (Montesano and Lopes 2009, 2012), continuous correlated beta processes for sharing evidence among Bernoulli experiments (Goetschalckx *et al.* 2011), Bayesian beta kernel models for binary classification and online learning (MacKenzie *et al.* 2014), and smooth beta-process methods for learning probability functions from Bernoulli tests (Rolland *et al.* 2019). Recent work on kernelized bandits with Bernoulli rewards further reinforces this distinction: probability-scale kernel learning is not a routine Gaussian-to-Bernoulli substitution, because GP-based estimators are not naturally constrained to the unit interval and Bernoulli-specific theoretical guarantees in kernelized settings remain an open problem (Mussi *et al.* 2024).

The **BKP** package (Zhao *et al.* 2026a), developed in R (R Core Team 2026) and available from the Comprehensive R Archive Network (CRAN), implements this probability-scale modeling strategy through the *Beta Kernel Process* (BKP). It combines kernel-weighted local likelihoods with beta-binomial conjugate updating. For each prediction input, neighboring observations contribute fractional success and failure counts through kernel weights, yielding closed-form conjugate posterior summaries for the local binomial probability. The package supports both individual binary responses and aggregated binomial counts. Its multinomial extension, the *Dirichlet Kernel Process* (DKP), uses the analogous Dirichlet-multinomial conjugate update to model categorical and multinomial responses. For larger datasets, the package also provides

TwinBKP and TwinDKP, twinning-based global-local approximations to the full BKP and DKP updates (Vakayil and Joseph 2022, 2024).

The term “Beta Kernel Process” is used descriptively in this article. It is distinct from the beta process in Bayesian nonparametrics (Hjort 1990), and is not to be confused with the kernel beta process for covariate-dependent feature learning (Ren *et al.* 2011). Here, “Beta” refers to the beta distribution used for local binomial probability summaries, “Kernel” refers to kernel-weighted sharing of evidence across inputs, and “Process” refers to the input-indexed collection of such beta distributions over the input space. The name is intended to emphasize a probability-scale analogue of kernel smoothing with an input-indexed family of probability distributions: BKP constructs an input-indexed family of conjugate beta posteriors directly on the probability scale, rather than placing a Gaussian-process prior on a latent real-valued function. In this article, BKP is therefore interpreted as a Bayesian-inspired, probability-scale kernel smoothing framework based on local-likelihood conjugate updating, rather than as a fully specified Bayesian stochastic-process model in the Gaussian-process sense.

Mature software libraries are available for GP modeling, including **DiceKriging** (Roustant *et al.* 2012), **GPfit** (MacDonald *et al.* 2015), and **gplite** (Piironen 2022) in R, and **GPyTorch** (Gardner *et al.* 2018), **GPflow** (Matthews *et al.* 2017), and **GPy** (GPy since 2012) in Python. In contrast, publicly available software dedicated to probability-scale beta and Dirichlet kernel modeling of binomial and multinomial responses remains limited. To our knowledge, **BKP** is the first publicly available R package dedicated to BKP and DKP methodology, providing a unified and extensible implementation for BKP, DKP, TwinBKP, and TwinDKP modeling. It includes model fitting, prediction, posterior simulation, visualization, prior specification, multiple kernel functions, loss-based hyperparameter tuning, effective-sample-size calibration, efficient C++ routines implemented through **Rcpp** (Eddelbuettel and François 2011) and **RcppArmadillo** (Eddelbuettel and Sanderson 2014), and standard S3 methods. These features make **BKP** suitable for probability-surface modeling tasks in biomedicine, ecology, social science, and industrial statistics.

The contribution of this article is twofold. Methodologically, it consolidates BKP and DKP under a common local-likelihood conjugate updating formulation, clarifies their Bayesian-inspired interpretation, and extends the framework with fixed and data-adaptive prior specifications, effective-sample-size calibration, compactly supported kernels, and twinning-based global-local approximations. Computationally, it provides a reproducible R implementation that integrates model fitting, prediction, simulation from fitted conjugate posteriors, visualization, hyperparameter tuning, and standard S3 methods for BKP, DKP, TwinBKP, and TwinDKP. The package provides a probability-scale workflow when closed-form conjugate posterior summaries, transparent local evidence borrowing, and direct binomial or multinomial updating are desirable.

The remainder of the paper is organized as follows. Section 2 presents the statistical foundation of BKP, DKP, and the TwinBKP/TwinDKP approximations. Section 3 describes the package interface, including fitting, prediction, simulation, visualization, and S3 methods. Section 4 provides illustrative examples for binomial and multinomial responses, together with GP-based comparisons and timing experiments. Section 5 demonstrates the workflow in two real-data applications. Section 6 concludes with limitations and future directions. Appendix A reports a simulation-based assessment of empirical pointwise interval coverage.

2. Statistical Foundation

2.1. Beta Kernel Process

Let $\mathbf{x} = (x_1, x_2, \dots, x_d) \in \mathcal{X} \subset \mathbb{R}^d$ denote a d -dimensional input. Suppose the success probability surface $\pi(\mathbf{x}) \in [0, 1]$ is unknown. At each location $\mathbf{x}$, the observed response is modeled as

$$y(\mathbf{x}) \sim \text{Binomial}(m(\mathbf{x}), \pi(\mathbf{x})),$$

where $y(\mathbf{x})$ is the number of successes out of $m(\mathbf{x})$ independent trials. The full dataset comprises n observations $\mathcal{D}_n = \{(\mathbf{x}_i, y_i, m_i)\}_{i=1}^n$, where we write $y_i = y(\mathbf{x}_i)$ and $m_i = m(\mathbf{x}_i)$ for brevity.

At each input location $\mathbf{x}$, the success probability is assigned a Beta prior,

$$\pi(\mathbf{x}) \sim \text{Beta}(\alpha_0(\mathbf{x}), \beta_0(\mathbf{x})),$$

where $\alpha_0(\mathbf{x}) > 0$ and $\beta_0(\mathbf{x}) > 0$ are input-dependent shape parameters. Details on prior specification are discussed in Section 2.3.

Let $k : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1]$ denote a user-defined kernel function measuring the similarity between input locations. Using the kernel-weighted local likelihood update, the BKP model constructs a closed-form conjugate posterior for $\pi(\mathbf{x})$ as

$$\begin{aligned} \pi(\mathbf{x}) \mid \mathcal{D}_n &\sim \text{Beta}(\alpha_n(\mathbf{x}), \beta_n(\mathbf{x})), \\ \alpha_n(\mathbf{x}) &= \alpha_0(\mathbf{x}) + \sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i) y_i = \alpha_0(\mathbf{x}) + \mathbf{k}^\top(\mathbf{x}) \mathbf{y}, \\ \beta_n(\mathbf{x}) &= \beta_0(\mathbf{x}) + \sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i) (m_i - y_i) = \beta_0(\mathbf{x}) + \mathbf{k}^\top(\mathbf{x}) (\mathbf{m} - \mathbf{y}), \end{aligned} \tag{1}$$

where $\mathbf{k}(\mathbf{x}) = [k(\mathbf{x}, \mathbf{x}_1), \dots, k(\mathbf{x}, \mathbf{x}_n)]^\top$ is the vector of kernel weights, $\mathbf{y} = (y_1, \dots, y_n)^\top$, and $\mathbf{m} = (m_1, \dots, m_n)^\top$ (Goetschalckx *et al.* 2011; Rolland *et al.* 2019).

The corresponding posterior predictive distribution for a future binomial response at $\mathbf{x}$ with known trial size $m(\mathbf{x})$ is

$$y(\mathbf{x}) \mid \mathcal{D}_n, m(\mathbf{x}) \sim \text{Beta-Binomial}(m(\mathbf{x}), \alpha_n(\mathbf{x}), \beta_n(\mathbf{x})). \tag{2}$$

Remark 1 (Bayesian interpretation). *For a fixed input location $\mathbf{x}$ and fixed kernel hyperparameters, the kernel-weighted local likelihood is*

$$\tilde{L}(\pi(\mathbf{x}); \mathcal{D}_n) \propto \prod_{i=1}^n \{\pi(\mathbf{x})^{y_i} (1 - \pi(\mathbf{x}))^{m_i - y_i}\}^{k(\mathbf{x}, \mathbf{x}_i)} = \pi(\mathbf{x})^{\mathbf{k}^\top(\mathbf{x}) \mathbf{y}} \{1 - \pi(\mathbf{x})\}^{\mathbf{k}^\top(\mathbf{x}) (\mathbf{m} - \mathbf{y})}.$$

Combining this local likelihood with the beta prior yields the conjugate beta posterior in (1). This construction is analogous to local likelihood estimation (Fan *et al.* 1998), where observations are reweighted according to their distance from the target input. Thus, although the BKP update leverages beta-binomial conjugacy, it differs from a fully specified Bayesian stochastic-process model. Moreover, kernel parameters are selected by empirical risk minimization rather than by posterior inference. BKP is therefore best interpreted as a Bayesian-inspired, probability-scale smoothing framework based on local-likelihood conjugate updating.

Remark 2 (Pointwise prediction cost). *For fixed kernel hyperparameters, computing the BKP conjugate posterior at a new input location requires $\mathcal{O}(n)$ kernel evaluations and weighted aggregations, because the update in (1) is available in closed form. The overall cost of model fitting, including LOOCV-based hyperparameter tuning, is discussed in Section 2.4.*

Based on the resulting conjugate posterior in (1), the posterior mean

$$\hat{\pi}_n(\mathbf{x}) = \mathbb{E}[\pi(\mathbf{x}) \mid \mathcal{D}_n] = \frac{\alpha_n(\mathbf{x})}{\alpha_n(\mathbf{x}) + \beta_n(\mathbf{x})} \quad (3)$$

serves as a smooth estimator of the latent success probability. The corresponding posterior variance

$$\sigma_n^2(\mathbf{x}) = \text{Var}[\pi(\mathbf{x}) \mid \mathcal{D}_n] = \frac{\hat{\pi}_n(\mathbf{x})\{1 - \hat{\pi}_n(\mathbf{x})\}}{\alpha_n(\mathbf{x}) + \beta_n(\mathbf{x}) + 1} \quad (4)$$

provides a local posterior measure of uncertainty under the fitted BKP update. These posterior summaries can be used to visualize the fitted probability surface and its local uncertainty across the input space, particularly in regions with sparse data coverage. See Section 4 for illustrations.

When the inferential target is the future binomial count rather than the latent success probability, the posterior predictive distribution in (2) gives

$$\mu_n(\mathbf{x}) = \mathbb{E}\{y(\mathbf{x}) \mid \mathcal{D}_n, m(\mathbf{x})\} = m(\mathbf{x})\hat{\pi}_n(\mathbf{x}).$$

The corresponding posterior predictive variance is

$$v_n^2(\mathbf{x}) = \text{Var}\{y(\mathbf{x}) \mid \mathcal{D}_n, m(\mathbf{x})\} = m(\mathbf{x})\{\alpha_n(\mathbf{x}) + \beta_n(\mathbf{x}) + m(\mathbf{x})\}\sigma_n^2(\mathbf{x}).$$

This predictive variance accounts for both uncertainty in the latent success probability and binomial sampling variability.

For binary classification, the posterior mean can be thresholded to produce hard prediction through

$$\hat{y}(\mathbf{x}) = \begin{cases} 1 & \text{if } \hat{\pi}_n(\mathbf{x}) > \pi_0, \\ 0 & \text{otherwise,} \end{cases} \quad (5)$$

where $\pi_0 \in (0, 1)$ is a user-specified threshold, typically set to be 0.5.

2.2. Effective-sample-size calibration

For a beta distribution, the sum of the two shape parameters controls the concentration of the distribution and is commonly interpreted as a posterior precision or an effective sample size (ESS) (Morita *et al.* 2008). For the BKP update in (1), the data-driven contribution to this precision at input location $\mathbf{x}$ is

$$m_K(\mathbf{x}) = \mathbf{k}(\mathbf{x})^\top \mathbf{m}.$$

When the binomial trial sizes m_i are highly heterogeneous, this kernel-weighted trial size may either overstate or understate the effective information available near $\mathbf{x}$. Consequently, posterior uncertainty summaries may be overly concentrated or overly diffuse relative to the

intended local amount of information. The purpose of ESS calibration is to rescale the kernel-weighted successes and failures by a common factor so that their total pseudo-count matches a target ESS, while leaving the kernel-weighted empirical success proportion unchanged. Here, calibration refers to matching the scale of the kernel-weighted pseudo-counts to a target effective sample size, rather than to a formal frequentist coverage guarantee.

To construct the target ESS, let $m_S(\mathbf{x})$ denote a continuous Shepard interpolation (Shepard 1968) of the observed trial sizes $m_1, \dots, m_n$. We then attenuate this interpolated trial size according to the proximity of $\mathbf{x}$ to the observed design points by defining

$$m_{\text{tar}}(\mathbf{x}) = \rho(\mathbf{x})m_S(\mathbf{x}), \quad \rho(\mathbf{x}) = \max_{1 \leq i \leq n} k(\mathbf{x}, \mathbf{x}_i).$$

At the observed design points, the Shepard interpolation is exact for distinct inputs. Together with $k(\mathbf{x}_i, \mathbf{x}_i) = 1$, this gives

$$m_{\text{tar}}(\mathbf{x}_i) = m_i, \quad i = 1, \dots, n.$$

Away from the observed inputs, $\rho(\mathbf{x})$ decreases as the maximum kernel similarity decreases, so that the target ESS is reduced in regions with weak support from the training data.

The ESS-calibrated BKP posterior is obtained by replacing the data contribution in (1) with

$$\alpha_n^{\text{ESS}}(\mathbf{x}) = \alpha_0(\mathbf{x}) + c(\mathbf{x})\mathbf{k}(\mathbf{x})^\top \mathbf{y}, \quad \beta_n^{\text{ESS}}(\mathbf{x}) = \beta_0(\mathbf{x}) + c(\mathbf{x})\mathbf{k}(\mathbf{x})^\top (\mathbf{m} - \mathbf{y}),$$

where $c(\mathbf{x}) = m_{\text{tar}}(\mathbf{x})/m_K(\mathbf{x})$ whenever $m_K(\mathbf{x}) > 0$. If $m_K(\mathbf{x}) = 0$, no training observation contributes to the update and the posterior is taken to be the baseline beta prior at $\mathbf{x}$. For $m_K(\mathbf{x}) > 0$, the adjusted data contribution satisfies

$$c(\mathbf{x})\mathbf{k}(\mathbf{x})^\top \mathbf{m} = m_{\text{tar}}(\mathbf{x}).$$

Thus, the calibration changes only the total pseudo-count contributed by the data, while preserving the kernel-weighted empirical success proportion. The empirical pointwise coverage of the resulting intervals is examined in Appendix A.

2.3. Prior Specification

The **BKP** package implements three prior specification strategies. These options differ in how the prior mean and prior precision are specified across the input space.

- **Non-informative prior:** The default choice is the uniform beta prior, with $\alpha_0(\mathbf{x}) = \beta_0(\mathbf{x}) \equiv 1$ for all $\mathbf{x}$. This option is appropriate when no external prior information is available and a weak baseline prior is desired.
- **Informative prior with fixed mean:** When prior information about the overall success probability $p_0 \in (0, 1)$ is available, a fixed informative prior can be specified by

$$\alpha_0(\mathbf{x}) = r_0 p_0, \quad \beta_0(\mathbf{x}) = r_0 (1 - p_0),$$

for all $\mathbf{x}$, where $r_0 > 0$ controls the prior precision. Larger values of r_0 imply a more concentrated prior distribution around p_0 . The uniform prior is recovered as the special case $r_0 = 2$ and $p_0 = 0.5$. When no external value of p_0 is available, a practical default is the empirical mean of the observed proportions, $\bar{p} = n^{-1} \sum_{i=1}^n y_i / m_i$.

- **Data-adaptive prior:** To accommodate variation over the input space, **BKP** also supports a data-adaptive prior in which both the prior mean and prior precision vary with $\mathbf{x}$. Let $S_K(\mathbf{x}) = \sum_{i=1}^n k(\mathbf{x}, \mathbf{x}_i)$ denote the local kernel mass. When $S_K(\mathbf{x}) > 0$, define normalized kernel weights by $w_i(\mathbf{x}) = k(\mathbf{x}, \mathbf{x}_i)/S_K(\mathbf{x})$. The adaptive prior uses

$$p(\mathbf{x}) = \sum_{i=1}^n w_i(\mathbf{x}) \frac{y_i}{m_i}, \quad r(\mathbf{x}) = r_0 S_K(\mathbf{x}),$$

and sets

$$\alpha_0(\mathbf{x}) = r(\mathbf{x})p(\mathbf{x}), \quad \beta_0(\mathbf{x}) = r(\mathbf{x})\{1 - p(\mathbf{x})\}.$$

This empirical-Bayes-type construction uses local kernel-weighted information to define a location-dependent baseline prior. In regions with stronger support from nearby observations, the prior becomes more concentrated; in regions with weaker support, it remains more diffuse. The goal is to stabilize the conjugate update under heterogeneous sampling density, rather than to impose a fixed prior strength uniformly over the input space. For numerical stability, the implementation uses small positive lower bounds for the local kernel mass and for the resulting beta shape parameters, ensuring valid prior parameters when the kernel mass is zero or nearly zero, as may occur with compactly supported kernels. This construction is in the spirit of local likelihood modeling (Fan and Gijbels 1996).

The choice of the global precision parameter r_0 depends on the response format and the intended role of the prior. For binary classification problems, where each input location typically contributes a single Bernoulli observation, a relatively small value such as $0.01 \leq r_0 \leq 0.1$ is often useful to avoid prior domination and keep the fitted probabilities primarily data-driven. This behavior is illustrated in Example 1 of Section 4.1. For aggregated binomial responses, a reasonable starting value is the mean number of trials per input location, which makes the adaptive prior strength comparable to the typical information content of one observed binomial count. In applications without reliable prior information, sensitivity analysis over a grid such as $r_0 \in \{0.001, 0.01, 0.1, 1, 5, 10\}$ can be used to assess the stability of fitted probability surfaces and decision boundaries.

2.4. Model Selection via Kernel Hyperparameter Tuning

Kernel Functions

Let $h(\mathbf{x}, \mathbf{x}'; \boldsymbol{\theta})$ denote the scaled Euclidean distance

$$h(\mathbf{x}, \mathbf{x}'; \boldsymbol{\theta}) = \sqrt{\sum_{j=1}^d \left(\frac{x_j - x'_j}{\theta_j} \right)^2},$$

where $\boldsymbol{\theta} = (\theta_1, \theta_2, \dots, \theta_d)$ contains positive kernel length-scale parameters. Smaller values of θ_j make the kernel more sensitive to differences in the j th input component, whereas larger values imply smoother borrowing along that dimension. The **BKP** package supports anisotropic kernels, where each input dimension has its own length-scale parameter, and isotropic kernels, where a single common length-scale is used, that is, $\theta_1 = \dots = \theta_d = \theta$.

Kernel Type	Function $k(h)$
Gaussian	$k(h) = \exp(-h^2)$
Matérn $\nu = 5/2$	$k(h) = \left(1 + \sqrt{5}h + \frac{5}{3}h^2\right) \exp(-\sqrt{5}h)$
Matérn $\nu = 3/2$	$k(h) = \left(1 + \sqrt{3}h\right) \exp(-\sqrt{3}h)$
Wendland	$k(h) = (\zeta h + 1) \max\{0, 1 - h\}^\zeta, \quad \zeta = \lfloor d/2 \rfloor + 3$

Table 1: Kernel functions implemented in the **BKP** package.

Based on this metric, the kernel function is written as $k(\mathbf{x}, \mathbf{x}') = k(h)$, where the functional form of $k(\cdot)$ determines the kernel type (Rasmussen and Williams 2006). The **BKP** package implements four kernel functions: Gaussian, Matérn 5/2, Matérn 3/2, and a compactly supported Wendland kernel (Wendland 1995), as summarized in Table 1.

Loss Functions

Kernel hyperparameters are selected by minimizing a user-specified loss function evaluated by leave-one-out cross-validation (LOOCV) (Montesano and Lopes 2012). For each data point $\mathbf{x}_i$, the prediction $\hat{\pi}_n^{-i}(\mathbf{x}_i; \boldsymbol{\theta})$ is computed with the i th observation excluded. In the implementation, this is achieved by setting the diagonal entries of the kernel matrix to zero, so that all leave-one-out predictions are obtained from a single kernel matrix rather than by refitting the model n times. The LOOCV procedure is known to mitigate overfitting and produce better generalization performance than marginal likelihood-based approaches (Rasmussen and Williams 2006; Vehtari *et al.* 2017).

The **BKP** package supports two loss functions: the Brier score and the log-loss. Let $\tilde{\pi}_i = y_i/m_i$ denote the empirical success proportion at input location $\mathbf{x}_i$.

Brier Score. The LOOCV Brier score is defined as

$$\text{BS}(\boldsymbol{\theta}; \mathcal{D}_n) = \frac{1}{n} \sum_{i=1}^n \left\{ \hat{\pi}_n^{-i}(\mathbf{x}_i; \boldsymbol{\theta}) - \tilde{\pi}_i \right\}^2. \quad (6)$$

It penalizes the squared deviation between the leave-one-out predicted probability and the observed empirical proportion.

Log-Loss. The LOOCV log-loss is defined as

$$\text{LL}(\boldsymbol{\theta}; \mathcal{D}_n) = -\frac{1}{n} \sum_{i=1}^n \left[\tilde{\pi}_i \log \hat{\pi}_n^{-i}(\mathbf{x}_i; \boldsymbol{\theta}) + (1 - \tilde{\pi}_i) \log \left\{ 1 - \hat{\pi}_n^{-i}(\mathbf{x}_i; \boldsymbol{\theta}) \right\} \right]. \quad (7)$$

This is the cross-entropy loss evaluated using the empirical proportions and the LOOCV predictive probabilities.

Remark 3 (Choice of loss function). *Although both criteria are evaluated through LOOCV, they emphasize different aspects of predictive performance (Gneiting and Raftery 2007; Flores*

et al. 2026). The Brier score penalizes squared deviations from empirical proportions and is relatively less sensitive to isolated extreme probability errors, making it a convenient choice for smooth probability-surface estimation with aggregated binomial data. In contrast, the log-loss corresponds to a cross-entropy criterion based on empirical proportions and penalizes overconfident mispredictions more strongly, especially when predicted probabilities are close to 0 or 1. In practice, the choice between the two depends on whether smoother probability estimation or classification-oriented probabilistic prediction is the primary objective.

Hyperparameter Optimization

To enforce positivity and improve numerical stability, kernel length-scale parameters are optimized on the logarithmic scale. Following the reparameterization strategy used in MacDonald et al. (2015), we define $\gamma_j = \log_{10}(\theta_j)$ for $j = 1, \dots, d_\gamma$, where $d_\gamma = 1$ for isotropic kernels and $d_\gamma = d$ for anisotropic kernels. Let $\boldsymbol{\gamma} = (\gamma_1, \dots, \gamma_{d_\gamma})$ denote the transformed parameter vector. For example, in the anisotropic Gaussian case,

$$k(\mathbf{x}, \mathbf{x}'; \boldsymbol{\gamma}) = \exp \left\{ - \sum_{j=1}^d \left(\frac{x_j - x'_j}{10^{\gamma_j}} \right)^2 \right\}. \quad (8)$$

The LOOCV loss functions in (6) and (7) can be non-convex in the kernel hyperparameters. To reduce dependence on a single initialization and seek a good approximation to the global minimum, **BKP** uses a multi-start derivative-free local optimization strategy. By default, the initial values are generated from a Latin hypercube design (LHD) with $n_0 = 10d_\gamma$ starting points (Loeppky et al. 2009). Each starting point is then refined using the SBPLX algorithm (Rowan 1990), as implemented in **nloptr** (Johnson 2007).

Following MacDonald et al. (2015), the initial search region for $\boldsymbol{\gamma}$ is

$$\Omega_0 = \left[\frac{\log_{10} d_\gamma - \log_{10} 500}{2}, \frac{\log_{10} d_\gamma + 2}{2} \right]^{d_\gamma}. \quad (9)$$

For all implemented kernels, including the Gaussian, Matérn, and Wendland kernels, the same initial search region is used to generate starting values. The subsequent local optimization is performed over the broader box $\Omega = [-3, 3]^{d_\gamma}$.

For each initial value, the optimizer solves

$$\hat{\boldsymbol{\gamma}}^{(i)} = \underset{\boldsymbol{\gamma} \in \Omega}{\operatorname{argmin}} L(\boldsymbol{\gamma}; \mathcal{D}_n), \quad i = 1, \dots, n_0,$$

where $L(\cdot)$ denotes either the Brier score in (6) or the log-loss in (7). The final estimate is selected as the best local solution:

$$i^* = \underset{1 \leq i \leq n_0}{\operatorname{argmin}} L(\hat{\boldsymbol{\gamma}}^{(i)}; \mathcal{D}_n), \quad \hat{\boldsymbol{\gamma}} = \hat{\boldsymbol{\gamma}}^{(i^*)}.$$

The fitted kernel length-scale parameters are then obtained by the inverse transformation $\hat{\theta}_j = 10^{\hat{\gamma}_j}$.

The optimization procedure is summarized as follows:

1. Generate $10d_\gamma$ initial values in Ω_0 using a space-filling LHD.

2. Run the SBPLX algorithm from each initial value over Ω .
3. Select the solution with the smallest LOOCV loss among all starts.

Remark 4 (Full-BKP fitting cost). *Compared with full GP inference, full BKP avoids the $\mathcal{O}(n^3)$ dense covariance-matrix factorization step by using kernel-weighted beta-binomial aggregation. For fixed input dimension, evaluating the BKP posterior at all training locations requires $\mathcal{O}(n^2)$ kernel evaluations and weighted aggregations. In the full-kernel implementation, this computation is carried out by constructing an $n \times n$ kernel matrix, which requires $\mathcal{O}(n^2)$ memory. Under LOOCV-based tuning, each loss evaluation uses one zero-diagonal kernel matrix to obtain all leave-one-out predictions simultaneously. If T denotes the total number of loss evaluations across all starts in the multi-start optimization, the overall computational cost of hyperparameter tuning is $\mathcal{O}(Tn^2)$, while the memory requirement remains $\mathcal{O}(n^2)$. Thus, full BKP avoids cubic GP-type matrix factorization but remains a quadratic full-kernel method.*

2.5. Scalable BKP via a Twinning-based Global-Local Approximation

For large datasets, the full BKP update in (1) may still be computationally demanding, as model fitting requires repeated construction of kernel matrices and each posterior prediction requires kernel evaluations against all n observations. To improve scalability, **BKP** implements a twinning-based global-local approximation, referred to as TwinBKP, inspired by the global-local design of TwinGP (Vakayil and Joseph 2024). Unlike TwinGP, which introduces the global-local structure through Gaussian process covariance matrices, TwinBKP adapts the idea to the conjugate beta-binomial updating structure of BKP.

Global-local subset construction

TwinBKP first selects a global subset

$$G \subset \{1, \dots, n\}, \quad |G| = g,$$

using the twinning algorithm (Vakayil and Joseph 2022). This subset is shared across all prediction locations and is intended to provide a representative summary of the observed training data. For a prediction location $\mathbf{x}$, TwinBKP further selects a local subset

$$L(\mathbf{x}) \subset \{1, \dots, n\} \setminus G, \quad |L(\mathbf{x})| = \ell,$$

consisting of the ℓ nearest neighbours of $\mathbf{x}$ among the remaining observations. The prediction-specific subset is then

$$U(\mathbf{x}) = G \cup L(\mathbf{x}).$$

For fixed input dimension, selecting the global subset by twinning has average cost $\mathcal{O}(n \log n)$, while constructing $L(\mathbf{x})$ requires an ℓ -nearest-neighbour query, with average cost $\mathcal{O}(\log n + \ell)$ per prediction location using the `nanoflann` kd-tree implementation (Blanco and Rai 2014). In the default implementation, **BKP** sets the global subset size to

$$g = \min\{n - 1, 50d, \max(\lfloor \sqrt{n} \rfloor, 10d)\}$$

and uses

$$\ell = \min\{n - g, \max(25, 3d)\}$$

nearest neighbours for the local subset at each prediction location. This rule allows the global subset to increase with the sample size for moderate n , while imposing a dimension-dependent upper cap and keeping the local refinement controlled for fixed input dimension.

Two-stage conjugate update

TwinBKP can be interpreted as a two-stage conjugate update. First, the global subset defines a global-informed intermediate beta distribution,

$$\alpha_G(\mathbf{x}) = \alpha_0(\mathbf{x}) + \sum_{i \in G} k_g(\mathbf{x}, \mathbf{x}_i; \boldsymbol{\theta}_g) y_i, \quad \beta_G(\mathbf{x}) = \beta_0(\mathbf{x}) + \sum_{i \in G} k_g(\mathbf{x}, \mathbf{x}_i; \boldsymbol{\theta}_g) (m_i - y_i),$$

where k_g is a global kernel chosen from the implemented kernel family in Table 1, typically a smooth Gaussian or Matérn kernel. Second, this global-informed distribution is updated using the local subset:

$$\begin{aligned} \pi(\mathbf{x}) \mid \mathcal{D}_U(\mathbf{x}) &\sim \text{Beta}(\alpha_U(\mathbf{x}), \beta_U(\mathbf{x})), \\ \alpha_U(\mathbf{x}) &= \alpha_G(\mathbf{x}) + \sum_{i \in L(\mathbf{x})} k_\ell(\mathbf{x}, \mathbf{x}_i; \theta_\ell) y_i, \\ \beta_U(\mathbf{x}) &= \beta_G(\mathbf{x}) + \sum_{i \in L(\mathbf{x})} k_\ell(\mathbf{x}, \mathbf{x}_i; \theta_\ell) (m_i - y_i), \end{aligned} \tag{10}$$

where k_ℓ is a local kernel, taken in the default implementation to be a compactly supported Wendland kernel. Thus, the global subset supplies broad background information shared across prediction locations, while the local subset refines the update near the target location. The baseline prior parameters $\alpha_0(\mathbf{x})$ and $\beta_0(\mathbf{x})$ follow the same specification strategies as in Section 2.3. Posterior summaries and predictive probabilities are then computed from the resulting beta posterior and the corresponding beta-binomial predictive distribution, as described in Section 2.1.

Kernel parameter specification

Kernel parameters in TwinBKP are handled separately for the global and local components. The global kernel parameter $\boldsymbol{\theta}_g$ is estimated by minimizing the same LOOCV-based loss as in Section 2.4, but only on the global subset

$$\mathcal{D}_G = \{(\mathbf{x}_i, y_i, m_i) : i \in G\}.$$

This avoids constructing the full $n \times n$ kernel matrix during tuning. Let T_g denote the total number of loss evaluations used in the global-subset optimization. The tuning cost is reduced from $\mathcal{O}(Tn^2)$ for full BKP to $\mathcal{O}(T_g g^2)$ for TwinBKP, and the kernel-matrix memory requirement is reduced from $\mathcal{O}(n^2)$ to $\mathcal{O}(g^2)$.

The scalar local range parameter θ_ℓ is not optimized separately. It is set to be the empirical covering radius (Vakayil and Joseph 2024) of the global subset on the normalized input scale,

$$\hat{\theta}_\ell = \max_{1 \leq i \leq n} \min_{j \in G} \|\mathbf{x}_i - \mathbf{x}_j\|_2.$$

This quantity is the fill distance (Fasshauer 2007) of the global subset relative to the empirical training design, measuring how well the global subset covers the observed training inputs. For

the compactly supported Wendland kernel, it provides a conservative support radius for local borrowing and avoids an additional local hyperparameter optimization step. Using a kd-tree built on the global subset, this empirical covering radius can be computed in $\mathcal{O}(n \log g)$ time for fixed input dimension.

Remark 5 (TwinBKP fitting and prediction cost). *For fixed input dimension, the overall computational cost of TwinBKP is driven by global subset selection, global-subset tuning, and prediction-specific local updates. With global subset size g and local subset size ℓ , the fitting cost is $\mathcal{O}(n \log n + T_g g^2)$. The corresponding memory requirement is $\mathcal{O}(n + g^2)$, including storage of the training inputs and the global kernel matrix. For prediction at t locations, the computational cost is $\mathcal{O}\{t(\log n + g + \ell)\}$, combining kd-tree local-neighbour queries with beta-binomial pseudo-count aggregation over the global and local subsets. Under the default scaling rule, g is controlled by both the sample size and the dimension-dependent cap $50d$, while ℓ is bounded by $\max(25, 3d)$ for fixed d . With fixed optimization budgets and fixed input dimension, the fitting cost is therefore dominated by the $\mathcal{O}(n \log n)$ global-subset construction term.*

2.6. Extension to Dirichlet Kernel Process

The BKP model extends naturally to multi-class responses through the Dirichlet Kernel Process (DKP), which replaces the binomial likelihood with a multinomial likelihood and the beta prior with a Dirichlet prior (MacKenzie *et al.* 2014).

Let the response at input $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^d$ be $\mathbf{y}(\mathbf{x}) = (y_1(\mathbf{x}), \dots, y_q(\mathbf{x}))$, where $y_s(\mathbf{x})$ denotes the count of class s and $m(\mathbf{x}) = \sum_{s=1}^q y_s(\mathbf{x})$ is the total number of trials. We assume

$$\mathbf{y}(\mathbf{x}) \sim \text{Multinomial}(m(\mathbf{x}), \boldsymbol{\pi}(\mathbf{x})),$$

where $\boldsymbol{\pi}(\mathbf{x}) = (\pi_1(\mathbf{x}), \dots, \pi_q(\mathbf{x}))$ is a vector of class probabilities satisfying $\sum_{s=1}^q \pi_s(\mathbf{x}) = 1$. At each input location, a Dirichlet prior is assigned to the class-probability vector,

$$\boldsymbol{\pi}(\mathbf{x}) \sim \text{Dirichlet}(\boldsymbol{\alpha}_0(\mathbf{x})),$$

where $\boldsymbol{\alpha}_0(\mathbf{x}) = (\alpha_{0,1}(\mathbf{x}), \dots, \alpha_{0,q}(\mathbf{x}))$ contains positive prior concentration parameters. Given training data $\mathcal{D}_n = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$, let $\mathbf{Y} \in \mathbb{R}^{n \times q}$ denote the response matrix whose i th row is $\mathbf{y}_i^\top$. The kernel-smoothed conjugate posterior is

$$\boldsymbol{\pi}(\mathbf{x}) \mid \mathcal{D}_n \sim \text{Dirichlet}(\boldsymbol{\alpha}_n(\mathbf{x})), \quad \boldsymbol{\alpha}_n(\mathbf{x}) = \boldsymbol{\alpha}_0(\mathbf{x}) + \mathbf{k}^\top(\mathbf{x})\mathbf{Y}. \quad (11)$$

The corresponding posterior predictive distribution for a future multinomial response at $\mathbf{x}$ with known trial size $m(\mathbf{x})$ is

$$\mathbf{y}(\mathbf{x}) \mid \mathcal{D}_n, m(\mathbf{x}) \sim \text{Dirichlet-Multinomial}(m(\mathbf{x}), \boldsymbol{\alpha}_n(\mathbf{x})). \quad (12)$$

Let $A_n(\mathbf{x}) = \sum_{s=1}^q \alpha_{n,s}(\mathbf{x})$. The posterior mean and marginal posterior variance of the class probability for class s are

$$\hat{\pi}_{n,s}(\mathbf{x}) = \frac{\alpha_{n,s}(\mathbf{x})}{A_n(\mathbf{x})}, \quad \sigma_{n,s}^2(\mathbf{x}) = \frac{\hat{\pi}_{n,s}(\mathbf{x})\{1 - \hat{\pi}_{n,s}(\mathbf{x})\}}{A_n(\mathbf{x}) + 1}, \quad s = 1, \dots, q.$$

For the future count of class s , the posterior predictive mean and marginal variance are

$$\mu_{n,s}(\mathbf{x}) = m(\mathbf{x})\hat{\pi}_{n,s}(\mathbf{x}), \quad v_{n,s}^2(\mathbf{x}) = m(\mathbf{x})\{A_n(\mathbf{x}) + m(\mathbf{x})\}\sigma_{n,s}^2(\mathbf{x}).$$

For classification tasks, the predicted label is assigned by the posterior predictive MAP rule,

$$\hat{y}(\mathbf{x}) = \operatorname{argmax}_{s \in \{1, \dots, q\}} \hat{\pi}_{n,s}(\mathbf{x}). \quad (13)$$

The prior specification strategies in Section 2.3 extend to DKP by replacing the beta shape parameters with Dirichlet concentration parameters. For example, a fixed prior can be specified through $\alpha_{0,s}(\mathbf{x}) = r_0 p_{0,s}$, where $\mathbf{p}_0 = (p_{0,1}, \dots, p_{0,q})$ is a class-probability vector and r_0 controls the total prior concentration. The non-informative and data-adaptive priors are constructed analogously by specifying baseline class proportions and a total concentration parameter over the input space.

Kernel hyperparameters are tuned by minimizing LOOCV-based loss functions, as described in Section 2.4. For multi-class responses, **BKP** supports the multi-class Brier score and log-loss,

$$\text{BS}_{\text{multi}}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \sum_{s=1}^q \left\{ \hat{\pi}_{n,s}^{-i}(\mathbf{x}_i; \boldsymbol{\theta}) - \tilde{\pi}_{i,s} \right\}^2, \quad (14)$$

$$\text{LL}_{\text{multi}}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n \sum_{s=1}^q \tilde{\pi}_{i,s} \log \hat{\pi}_{n,s}^{-i}(\mathbf{x}_i; \boldsymbol{\theta}), \quad (15)$$

where $\tilde{\pi}_{i,s} = y_{i,s}/m_i$ denotes the empirical class proportion and $m_i = \sum_{s=1}^q y_{i,s}$. When $q = 2$, the log-loss reduces to the binary log-loss in (7), while the multi-class Brier score is proportional to the binary Brier score in (6); this constant factor does not affect hyperparameter selection.

The ESS calibration in Section 2.2 also extends to DKP by calibrating the total Dirichlet concentration. Let $m_K(\mathbf{x}) = \mathbf{k}(\mathbf{x})^\top \mathbf{m}$ and use the same target ESS $m_{\text{tar}}(\mathbf{x}) = \rho(\mathbf{x})m_S(\mathbf{x})$. For $m_K(\mathbf{x}) > 0$, define $c(\mathbf{x}) = m_{\text{tar}}(\mathbf{x})/m_K(\mathbf{x})$. The ESS-calibrated DKP posterior is then

$$\boldsymbol{\alpha}_n^{\text{ESS}}(\mathbf{x}) = \boldsymbol{\alpha}_0(\mathbf{x}) + c(\mathbf{x})\mathbf{k}^\top(\mathbf{x})\mathbf{Y}.$$

As in the BKP case, if $m_K(\mathbf{x}) = 0$, no training observation contributes to the update and the posterior is taken to be the baseline Dirichlet prior at $\mathbf{x}$.

Finally, the twinning-based global-local approximation extends directly to DKP. The resulting TwinDKP model uses the same global-local subset construction as TwinBKP, but replaces the beta pseudo-count updates with Dirichlet pseudo-count vector updates over the global and local subsets.

3. BKP Package

The **BKP** package (Zhao *et al.* 2026a) implements the Beta Kernel Process (BKP) for binary and binomial response data, together with its multinomial extension, the Dirichlet Kernel Process (DKP). It also provides scalable global-local variants, namely TwinBKP and TwinDKP, for applications where full kernel-matrix construction is computationally demanding. The

	Description
Functions	
<code>fit_BKP()</code>	Fit a Beta Kernel Process (BKP) model for binary/binomial data.
<code>fit_DKP()</code>	Fit a Dirichlet Kernel Process (DKP) model for categorical or multinomial data.
<code>fit_TwinBKP()</code>	Fit a scalable global-local TwinBKP approximation for binary/binomial data.
<code>fit_TwinDKP()</code>	Fit a scalable global-local TwinDKP approximation for categorical or multinomial data.
Core S3 Methods	
<code>predict()</code>	Compute posterior predictions at training or new input locations.
<code>simulate()</code>	Generate posterior simulations from fitted model objects.
<code>summary()</code>	Summarize model settings and posterior quantities.
<code>plot()</code>	Visualize fitted probability surfaces and posterior uncertainty.
<code>print()</code>	Print fitted models, predictions, simulations, and summaries.

Table 2: Main functions and S3 methods provided in the **BKP** package. The S3 methods apply to fitted model objects returned by the corresponding fitting functions. Additional accessor methods, such as `fitted()`, `parameter()`, and `quantile()`, are also available.

package follows the standard R modeling workflow, with separate functions for model fitting, posterior prediction, posterior simulation, visualization, and model summarization. It also exposes utilities for kernel construction, prior specification, and loss-based hyperparameter tuning, allowing users to adjust the kernel family, prior structure, loss criterion, and scalability settings according to the data-analysis task.

The main user-facing functions and S3 methods are summarized in Table 2. The package can be installed from CRAN using `install.packages("BKP")`; the development version is available from GitHub via `pak::pak("Jiangyan-Zhao/BKP")`. An interactive overview of the methodology, package workflow, examples, and related project resources is available from the **BKP** [project website](#).

3.1. BKP Model

We begin with the full BKP interface, which provides the baseline workflow for model fitting, posterior prediction, posterior simulation, visualization, and numerical extraction. The DKP model in Section 3.2 follows the same workflow but differs in the response format, posterior distribution, and multi-class prediction summaries. The TwinBKP and TwinDKP models in Section 3.3 retain the corresponding BKP and DKP response formats, while replacing full kernel aggregation with the scalable global-local approximation described in Section 2.5.

Model Fitting

The function `fit_BKP()` fits the BKP model in (1) for binary or aggregated binomial response data. Its main arguments are

```
fit_BKP(X, y, m, Xbounds = NULL,
```

```
prior = "noninformative", r0 = 2, p0 = mean(y / m),
kernel = "gaussian", isotropic = TRUE, theta = NULL,
loss = "brier", ess = "none", n_multi_start = NULL, n_threads = 1)
```

The arguments `X`, `y`, and `m` specify the data. The input `X` is an $n \times d$ numeric matrix, `y` is a length- n vector of observed successes, and `m` is the corresponding vector of binomial trial sizes. The optional argument `Xbounds` is a $d \times 2$ matrix containing the lower and upper bounds of each input dimension. When `Xbounds` is supplied, the input matrix is rescaled to the unit hypercube $[0, 1]^d$ before fitting. When `Xbounds = NULL`, `X` is assumed to have already been normalized to $[0, 1]^d$.

The arguments `prior`, `r0`, and `p0` define the prior specification for the latent success probability surface. The argument `prior` selects one of the prior forms described in Section 2.3: "noninformative", "fixed", or "adaptive". The prior parameters are constructed internally by `get_prior()`. The arguments `r0` and `p0` specify the prior precision and prior mean when they are required by the selected prior form.

The arguments `kernel`, `isotropic`, and `theta` determine the kernel family and length-scale structure. The argument `kernel` selects one of the implemented kernels, namely "gaussian", "matern52", "matern32", or "wendland". If `isotropic = TRUE`, a single common length-scale is used for all input dimensions; if `isotropic = FALSE`, dimension-specific length-scales are used. The argument `theta` can be supplied to fix the kernel length-scale parameter directly. In the isotropic case, `theta` must be a scalar; in the anisotropic case, it may be either a scalar, which is expanded to all dimensions, or a vector of length d . When `theta = NULL`, the length-scale parameter is selected by leave-one-out cross-validation.

The remaining arguments control model tuning, effective-sample-size calibration, and computation. The argument `loss` specifies the LOOCV criterion used for kernel hyperparameter tuning, either "brier" or "log_loss", as implemented in `loss_fun()`. The argument `ess` controls optional effective-sample-size calibration. The default `ess = "none"` uses the standard BKP posterior update, whereas `ess = "shepard"` rescales the kernel-weighted data contribution according to the Shepard-interpolation target described in Section 2.2. The argument `n_multi_start` specifies the number of initial values used in multi-start optimization; when it is `NULL`, the default is $10d_\gamma$, where $d_\gamma = 1$ for isotropic kernels and $d_\gamma = d$ for anisotropic kernels, as in Section 2.4.3. The argument `n_threads` specifies the number of parallel threads used during multi-start optimization when thread-level parallelization is available.

The helper functions `kernel_matrix()`, `get_prior()`, and `loss_fun()` are primarily used internally by `fit_BKP()`. They are exported so that advanced users can inspect the main computational components of the fitted model: `kernel_matrix()` constructs kernel-weight matrices for specified inputs and length-scale parameters, `get_prior()` constructs the prior beta parameters, and `loss_fun()` evaluates the LOOCV loss for a given kernel specification. These functions also support customized workflows built on the same BKP computational structure.

The function `fit_BKP()` returns an object of class 'BKP'. The object stores the fitted kernel specification, selected length-scale parameter, prior specification, normalized training inputs, observed binomial responses, and posterior beta parameters evaluated at the training inputs. These stored components are used by downstream S3 methods for prediction, simulation, visualization, and model summarization. When ESS calibration is used, the object also

records the calibration method and associated diagnostics.

The resulting ‘BKP’ object can be passed as the `object` argument to `predict()`, `simulate()`, and `summary()`, or as the `x` argument to `plot()` and `print()`.

Methods for the Fitted BKP Objects

For a fitted ‘BKP’ object `BKPmodel`, posterior prediction is obtained by

```
predict(BKPmodel, Xnew = NULL, CI_level = 0.95, threshold = 0.5,
        type = "probability", Mnew = NULL, ...)
```

By default, `type = "probability"` returns posterior summaries for the latent success probability $\pi(\mathbf{x})$. For each input location $\mathbf{x}$ in `Xnew`, or at the training locations when `Xnew = NULL`, the returned object contains the posterior mean $\hat{\pi}_n(\mathbf{x})$, posterior variance $\sigma_n^2(\mathbf{x})$, lower and upper bounds of the `CI_level` credible interval, and the posterior beta parameters $\alpha_n(\mathbf{x})$ and $\beta_n(\mathbf{x})$. If the fitted model corresponds to binary classification data, that is, all training trial sizes satisfy `m = 1`, the prediction object also includes a binary class label obtained by comparing $\hat{\pi}_n(\mathbf{x})$ with the specified `threshold`, following the rule in (5).

Alternatively, setting `type = "count"` returns posterior predictive summaries for a future binomial count. In this case, `Mnew` specifies the future trial size at each prediction location, either as a scalar common to all locations or as a vector with length equal to the number of prediction locations. The returned mean, variance, and interval bounds are then computed on the count scale under the beta-binomial posterior predictive distribution in (2).

The function call

```
simulate(BKPmodel, nsim = 1, seed = NULL, Xnew = NULL, threshold = NULL, ...)
```

generates pointwise posterior draws of the latent success probability from a fitted ‘BKP’ model. For each input location in `Xnew`, or at the training locations when `Xnew = NULL`, the method obtains the corresponding posterior beta parameters and independently draws `nsim` samples from $\text{Beta}(\alpha_n(\mathbf{x}), \beta_n(\mathbf{x}))$. The returned `samples` component is a matrix with n_{new} rows and `nsim` columns, where each column contains a collection of independent pointwise draws from the marginal beta posteriors evaluated at the supplied input locations. Thus, these draws should not be interpreted as realizations from a jointly dependent posterior process; in particular, duplicate input locations correspond to independent draws from the same marginal posterior distribution. If `threshold` is supplied, the simulated probabilities are additionally converted to binary class labels by thresholding each pointwise draw, and the optional `seed` argument sets the random seed for reproducibility. These draws are useful for visualizing marginal posterior uncertainty and for pointwise randomized decision rules. However, because the current BKP construction does not specify cross-location posterior dependence, they do not constitute coherent posterior sample paths or random surfaces. Consequently, downstream procedures requiring function-level posterior samples, such as standard Thompson sampling over a continuous probability surface (Garnett 2023), would require an additional joint dependence model.

The function call

```
summary(BKPmodel, ...)
```

returns a structured summary of a fitted ‘BKP’ object. The summary includes the number of training observations, input dimensionality, kernel family, isotropic setting, selected kernel hyperparameters, loss criterion, minimum achieved loss value, and prior specification. It also reports posterior summaries at the training inputs, including the posterior means and variances of the latent success probabilities. Thus, `summary()` provides a compact diagnostic view of both the fitted model configuration and the main posterior quantities used in subsequent prediction and visualization.

The `plot()` method provides graphical visualization of a fitted ‘BKP’ object, with the display determined by the selected input dimension(s). For one-dimensional displays, it plots the posterior mean of the latent success probability as a curve, together with a shaded credible interval and the observed proportions $\tilde{\pi}_i = y_i/m_i$. For binary classification data, the classification threshold can also be shown to indicate the induced decision boundary. For two-dimensional displays, the method evaluates the fitted model on a regular prediction grid and produces contour plots of posterior summaries. By default, non-classification plots include the posterior mean, posterior variance, and posterior interval bounds for the latent success probability; for classification data, the posterior mean and variance surfaces of the class probability are displayed. The argument `only_mean = TRUE` restricts the display to the posterior mean surface. For inputs with more than two dimensions, users specify one or two coordinates to visualize through the `dims` argument.

The **BKP** package uses S3 classes to organize fitted models and downstream outputs. A fitted ‘BKP’ object has a dedicated `print()` method for concise console display, while the outputs of `predict()`, `simulate()`, and `summary()` are returned as objects of classes ‘`predict_BKP`’, ‘`simulate_BKP`’, and ‘`summary_BKP`’, respectively. These classes provide method-specific printed summaries and separate the user-facing display from the underlying list structure, making the fitted objects easier to inspect while preserving access to their numerical components.

In addition to the core methods in Table 2, the **BKP** package provides accessor methods for extracting numerical components from fitted objects. For a fitted ‘BKP’ object, `fitted()` returns the posterior mean success probabilities at the training inputs, `parameter()` returns the fitted kernel length-scale parameter together with the posterior beta parameters $\alpha_n(\mathbf{x})$ and $\beta_n(\mathbf{x})$, and `quantile()` computes posterior quantiles of the latent success probability, with the requested quantile levels specified by `probs`. These accessors provide direct programmatic access to the fitted posterior summaries without relying on printed output.

3.2. DKP Model

The function `fit_DKP()` implements the DKP model in (11), extending the BKP framework from binary or binomial responses to categorical or aggregated multinomial response data. Its main arguments are

```
fit_DKP(X, Y, Xbounds = NULL,
        prior = "noninformative", r0 = 2, p0 = NULL,
        kernel = "gaussian", isotropic = TRUE, theta = NULL,
        loss = "brier", ess = "none", n_multi_start = NULL, n_threads = 1)
```

The main difference from `fit_BKP()` is the response format. Instead of separate vectors `y` and `m`, the DKP interface uses an $n \times q$ count matrix `Y`, where each row corresponds to one input location and each column to one response category. The multinomial trial size is defined by

the row sum of Y , so a separate m argument is not required. When `prior = "fixed"`, p_0 is a class-probability vector of length q ; if $p_0 = \text{NULL}$, it is set to the empirical class-proportion vector `colMeans(Y / rowSums(Y))`.

The function returns an object of class ‘DKP’, structurally parallel to a ‘BKP’ object. It stores the fitted kernel specification, selected length-scale parameter, prior specification, normalized training inputs, observed multinomial counts, and posterior Dirichlet concentration parameters evaluated at the training inputs. The key distinction is that the posterior parameter at an input $\mathbf{x}$ is a vector $\alpha_n(\mathbf{x})$, rather than a pair of beta shape parameters.

The downstream S3 methods follow the same workflow as for ‘BKP’ objects. The `predict()` method returns per-class posterior means, marginal variances, and marginal credible intervals for the latent class-probability vector. If each row of Y sums to one, it also returns predicted class labels using the MAP rule in (13). Setting `type = "count"` instead gives per-class marginal posterior predictive summaries for future multinomial counts with trial sizes specified by M_{new} . The `simulate()` method draws posterior samples from $\text{Dirichlet}\{\alpha_n(\mathbf{x})\}$ and, for single-label classification data, can also produce sampled class labels. The `plot()` method displays class-specific posterior summaries, using curves for one-dimensional inputs and contour plots for two-dimensional displays.

3.3. TwinBKP and TwinDKP Models

For larger datasets, the package provides the scalable variants `fit_TwinBKP()` and `fit_TwinDKP()`. TwinBKP corresponds to the global-local approximation described in Section 2.5, and TwinDKP uses the analogous Dirichlet pseudo-count update described at the end of Section 2.6. These functions follow the same response conventions as `fit_BKP()` and `fit_DKP()`, respectively: `fit_TwinBKP()` uses X , y , and m , whereas `fit_TwinDKP()` uses X and the multinomial count matrix Y . Their additional arguments control the global-local approximation and global-subset tuning, including the global and local kernels `global_kernel` and `local_kernel`, the global and local length-scale parameters `theta_g` and `theta_l`, the global subset size `g`, the local-neighbour size `l`, the number of twinning runs `twins`, and the diagnostic option `store_kernel`. In the current implementation, `global_kernel` can be selected from the implemented kernel family, whereas `local_kernel` is restricted to “wendland”, corresponding to the compactly supported local kernel used in the TwinBKP and TwinDKP approximations. The arguments `loss`, `n_multi_start`, `isotropic`, and `n_threads` play the same roles as in the full models, but are applied to global-subset tuning.

The returned objects, of classes ‘TwinBKP’ and ‘TwinDKP’, are designed to be used with the same S3 workflow as full BKP and DKP objects. In particular, the methods `predict()`, `simulate()`, `summary()`, `plot()`, `print()`, `fitted()`, `parameter()`, and `quantile()` are available for the fitted Twin objects. By default, the Twin implementations avoid storing dense $n \times n$ kernel matrices; setting `store_kernel = TRUE` stores diagnostic kernel matrices for testing and inspection, at the cost of increased memory usage. Effective-sample-size calibration is currently implemented for the full BKP and DKP models, whereas TwinBKP and TwinDKP use the uncalibrated global-local posterior updates to preserve the scalable approximation.

4. Examples using BKP

4.1. BKP Model

This subsection presents detailed illustrative examples based on the full BKP model. Examples for the DKP model and the TwinBKP/TwinDKP scalable approximations are provided in Sections 4.2 and 4.3, respectively.

Example 1 Let $x \in [-2, 2]$, and suppose the true Bernoulli probability function is given by

$$\pi_1(x) = \frac{1}{1 + e^{-3x}}, \quad (16)$$

which is referred as the function `true_pi_fun` in the code below. We aim to fit the BKP model based on seven input locations that are uniformly distributed over $[-2, 2]$, with each location associated with a binomial observation having a maximum trial count of 100. The input locations are generated using the `lhs()` function from the R package **tgpp** (Gramacy and Taddy 2010). The following R code illustrates how to simulate the data and fit the BKP model using the `fit_BKP()` function.

```
R> n <- 7
R> Xbounds <- matrix(c(-2,2), nrow = 1)
R> X <- lhs(n = n, rect = Xbounds)
R> true_pi <- true_pi_fun(X)
R> m <- sample(100, n, replace = TRUE)
R> y <- rbinom(n, size = m, prob = true_pi)
R> BKP_model_1D_1 <- fit_BKP(X, y, m, Xbounds = Xbounds)
```

The estimates of the parameters of the fitted BKP model can be displayed using the `print()` function:

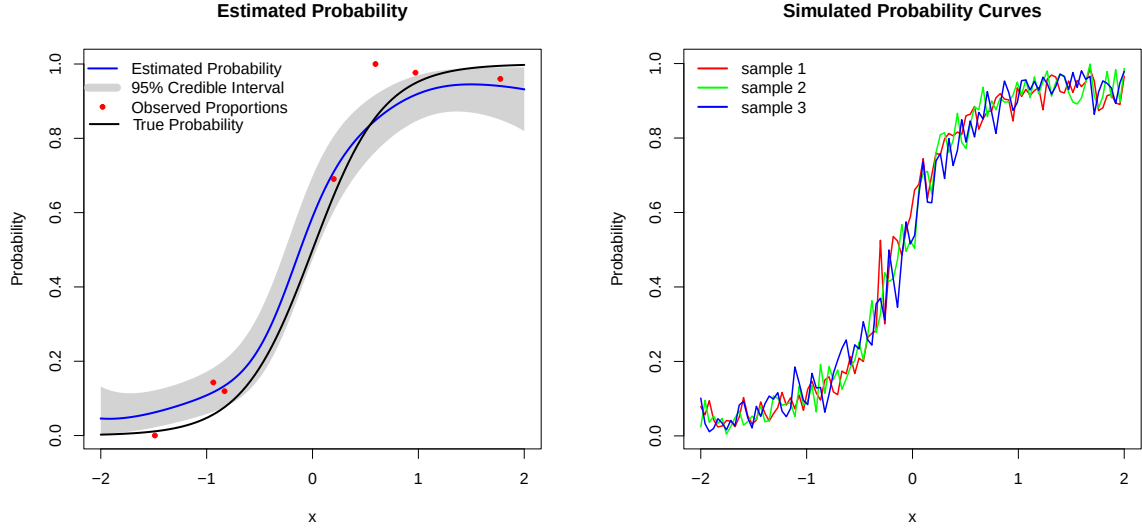
```
R> print(BKP_model_1D_1)
```

Beta Kernel Process (BKP) Model

```
Number of observations (n): 7
Input dimensionality (d): 1
Kernel type: (isotropic) gaussian
Optimized kernel parameters: 0.1748
Minimum achieved loss: 0.01165
Loss function: brier
Prior type: noninformative
```

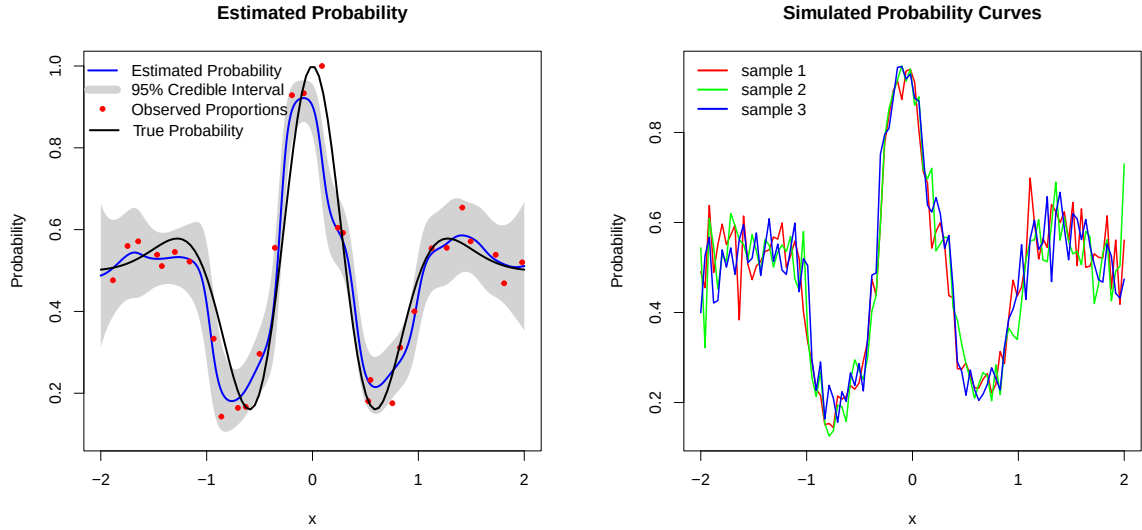
The `BKP_model_1D_1` object can be used for visualization and prediction over a grid of input values via the `plot()` and `simulate()` methods:

```
R> plot(BKP_model_1D_1)
R> Xnew <- matrix(seq(-2, 2, length = 100), ncol = 1)
R> sim <- simulate(BKP_model_1D_1, nsim = 3, Xnew = Xnew)
```



(a) Posterior mean and 95% CI of Example 1

(b) Pointwise posterior draws for Example 1



(c) Posterior mean and 95% CI of Example 2

(d) Pointwise posterior draws for Example 2

Figure 1: Posterior inference and pointwise simulation results from the fitted BKP models. In panels (b) and (d), each displayed curve connects independent draws from the marginal beta posteriors at the plotted input locations for visualization; the connected curves should not be interpreted as realizations from a jointly dependent posterior process.

Figure 1 presents two views of the model output. Panel (a) shows the posterior mean estimate of the probability function $\pi(x)$ (blue), along with a 95% credible interval (gray band), observed proportions (red dots), and the true underlying probability function (black). Panel (b) presents three posterior sample curves generated using the `simulate()` method, illustrating the variability of the estimated probability surface.

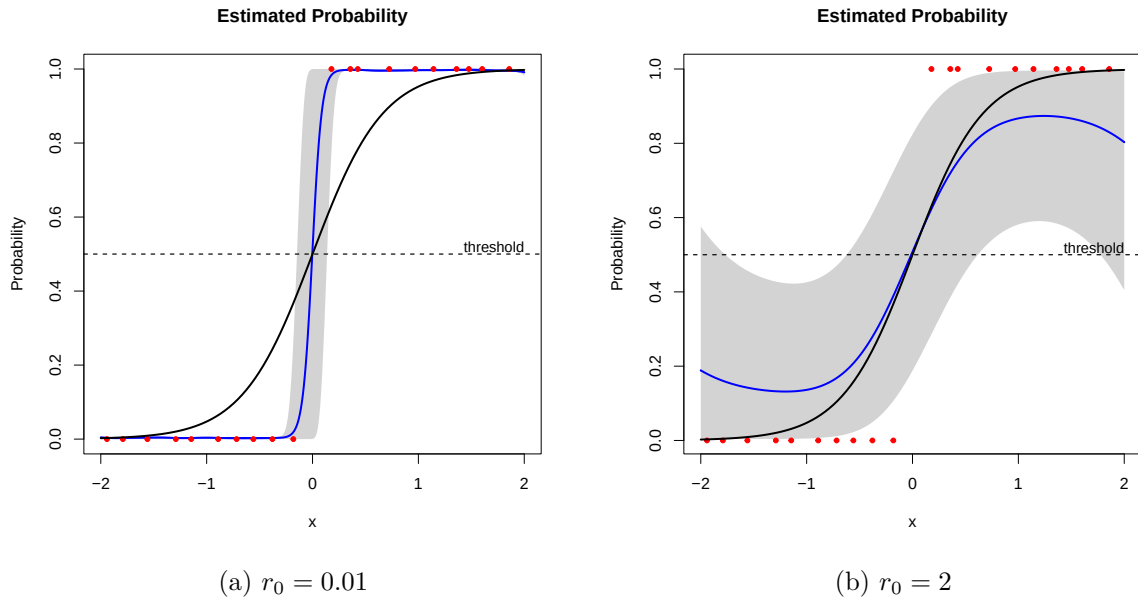


Figure 2: Posterior mean and 95% CI of Example 1 (classification task) under $r_0 = 0.01$ and 2

We continue with the same probability function of this example to show the impact of the global precision parameters, r_0 , in classification tasks. However, the sample size is changed from 7 to 20 for classification. The following R code generates the label of response and fit the BKP model for classification with r_0 being 0.01 and 2.

```
R> n <- 20
R> Xbounds <- matrix(c(-2, 2), nrow = 1)
R> X <- lhs(n = n, rect = Xbounds)
R> true_pi <- true_pi_fun(X)
R> m <- rep(1, n)
R> y <- as.numeric(true_pi > 0.5)
R> # Fit BKP model with r0 = 0.01
R> BKP_model_1D_1_class_1 <- fit_BKP(
+   X, y, m, Xbounds = Xbounds,
+   prior = "fixed", r0 = 0.01, loss = "log_loss")
R> # Fit BKP model with r0 = 2
R> BKP_model_1D_1_class_2 <- fit_BKP(
+   X, y, m, Xbounds = Xbounds,
+   prior = "fixed", r0 = 2, loss = "log_loss")
```

Figure 2a shows a sigmoidal curve with steep slope around zero (in solid line) and a narrow 95% credible interval (in grey band) under $r_0 = 0.01$, indicating a decisive and confident classification boundary. In contrast, Figure 2b displays a sine-shape curve with a much wider credible interval, reflecting greater uncertainty and less effective separation. This illustrates that a smaller value of r_0 can be preferable in this classification example.

Example 2 The first example is essentially a generalized linear model with a smooth logit link, and thus poses limited modeling complexity. To demonstrate the capability of the BKP model in handling more challenging probability surfaces, we consider a second example with a highly nonlinear underlying probability surface. Define the true Bernoulli probability as

$$\pi_2(x) = \frac{1}{2} \left[1 + e^{-x^2} \cos \left(10 \frac{1 - e^{-x}}{1 + e^{-x}} \right) \right], \quad (17)$$

where $x \in [-2, 2]$ (Goetschalckx *et al.* 2011). This example involves rapid local oscillations and strong nonlinearity, making it substantially more difficult to fit than Example 1. Here, we increase the number of locations to 30.

```
R> n <- 30
R> Xbounds <- matrix(c(-2,2), nrow = 1)
R> X <- lhs(n = n, rect = Xbounds)
R> true_pi <- true_pi_fun(X)
R> m <- sample(100, n, replace = TRUE)
R> y <- rbinom(n, size = m, prob = true_pi)
R> BKP_model_1D_2 <- fit_BKP(X, y, m, Xbounds = Xbounds)
```

The results are shown in panels (c) and (d) of Figure 1. Panel (c) shows that the BKP posterior mean follows the main nonlinear features of the probability function, while panel (d) illustrates posterior variability through three representative realizations of $\pi(x)$.

Example 3 We now consider a two-dimensional test function to further illustrate the modeling capabilities of the **BKP** package. Let $\mathbf{x} \in [0, 1]^2$, and define the latent surface using a re-scaled version of the Goldstein–Price function (Picheny *et al.* 2013):

$$f(\mathbf{x}) = \frac{\log[\{1 + a(\mathbf{x})\}\{30 + b(\mathbf{x})\}] - 8.6928}{2.4269}, \quad \text{with}$$

$$a(\mathbf{x}) = (4x_1 + 4x_2 - 3)^2 \times$$

$$\{75 - 56(x_1 + x_2) + 3(4x_1 - 2)^2 + 6(4x_1 - 2)(4x_2 - 2) + 3(4x_2 - 2)^2\},$$

$$b(\mathbf{x}) = (8x_1 - 12x_2 + 2)^2 \times$$

$$\{-14 - 128x_1 + 12(4x_1 - 2)^2 + 192x_2 - 36(4x_1 - 2)(4x_2 - 2) + 27(4x_2 - 2)^2\}.$$

The true Bernoulli probability surface is then defined by

$$\pi_3(\mathbf{x}) = \Phi\{f(\mathbf{x})\}, \quad (18)$$

where $\Phi(\cdot)$ is the cumulative distribution function of the standard normal distribution. This formulation produces a smooth yet highly non-linear response surface, providing a challenging test scenario for probabilistic modeling.

To construct the training data, we generate a LHD of size 100 over $[0, 1]^2$. Each location is associated with a binomial observation whose number of trials is randomly drawn from $\{1, \dots, 100\}$. The following R code demonstrates the data simulation and model fitting using the `fit_BKP()` function:

```

R> n <- 100
R> Xbounds <- matrix(c(0, 0, 1, 1), nrow = 2)
R> X <- lhs(n = n, rect = Xbounds)
R> true_pi <- true_pi_fun(X)
R> m <- sample(100, n, replace = TRUE)
R> y <- rbinom(n, size = m, prob = true_pi)
R> BKP_model_2D <- fit_BKP(X, y, m, Xbounds = Xbounds)
R> print(BKP_model_2D)

```

Beta Kernel Process (BKP) Model

```

Number of observations (n): 100
Input dimensionality (d): 2
Kernel type: (isotropic) gaussian
Optimized kernel parameters: 0.0805
Minimum achieved loss: 0.01125
Loss function: brier
Prior type: noninformative

```

Figure 3 is produced by the following call and displays the true probability surface together with the BKP posterior summaries.

```
R> plot(BKP_model_2D)
```

The results of model prediction are as follows:

```

R> Xnew <- lhs(n = 10, rect = Xbounds)
R> predict(BKP_model_2D, Xnew)

```

Prediction results on new data (Xnew).
Total number of prediction points: 10

```

Preview of predictions for new data (first 6 of 10 points):
      x1      x2  mean variance 2.5% quantile 97.5% quantile
0.2184 0.1046 0.3513  0.0042      0.2298      0.4836
0.9401 0.6108 0.1558  0.0043      0.0512      0.3043
0.6782 0.8087 0.5144  0.0021      0.4254      0.6031
0.1591 0.9120 0.9452  0.0006      0.8906      0.9817
0.7838 0.2556 0.5136  0.0033      0.4005      0.6260
0.4185 0.5215 0.2639  0.0010      0.2047      0.3277
...

```

We continue with Example 3 to examine the computational scaling of the BKP model in comparison with the logistic Gaussian process (LGP) model ([Rasmussen and Williams 2006](#)). The sample size ranges from 200 to 5000. Two settings for kernel hyperparameter specification are considered. In the first setting, the BKP kernel length-scale parameter is fixed at `theta =`

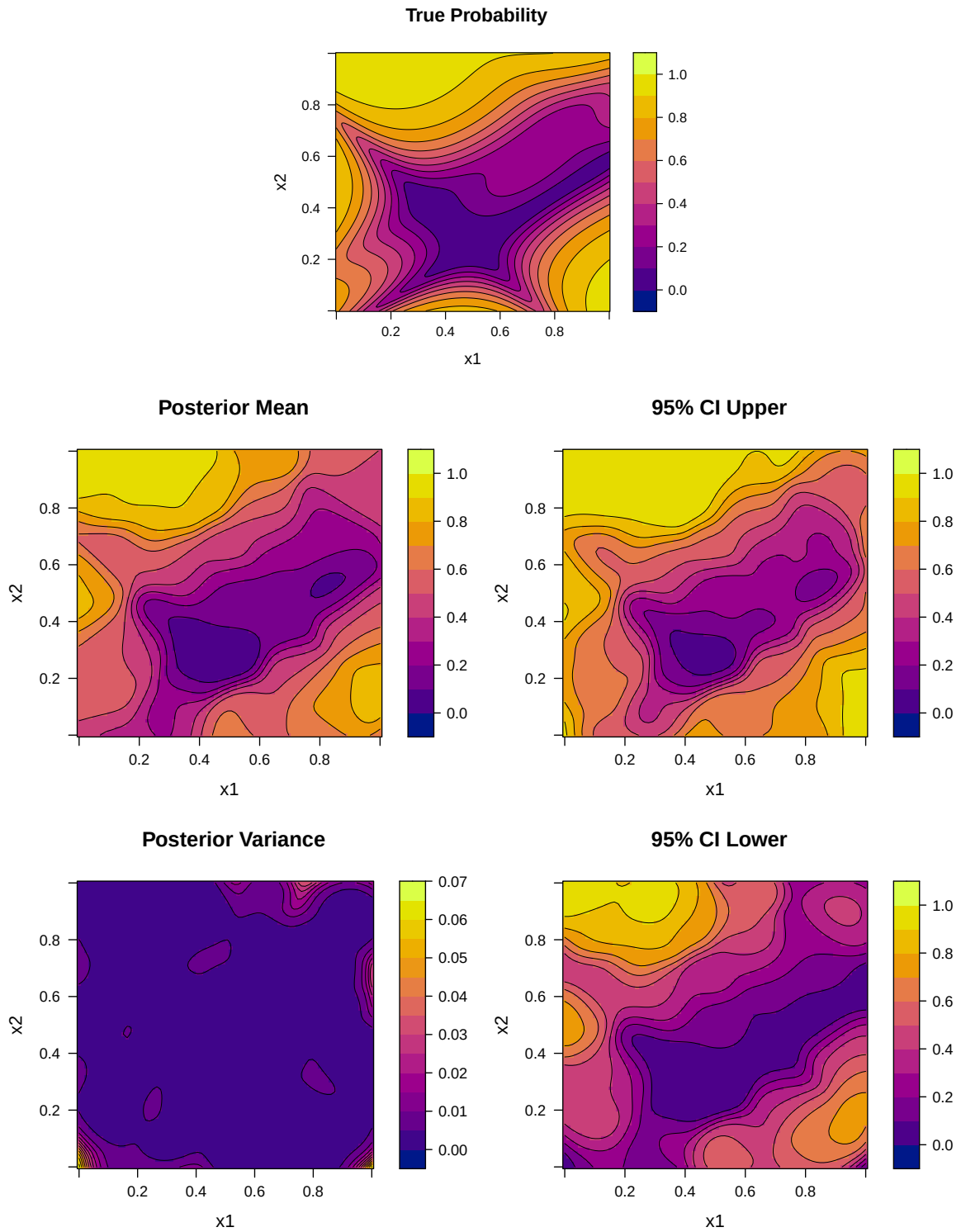


Figure 3: Comparison of the true probability surface and BKP-based posterior summaries of Example 3

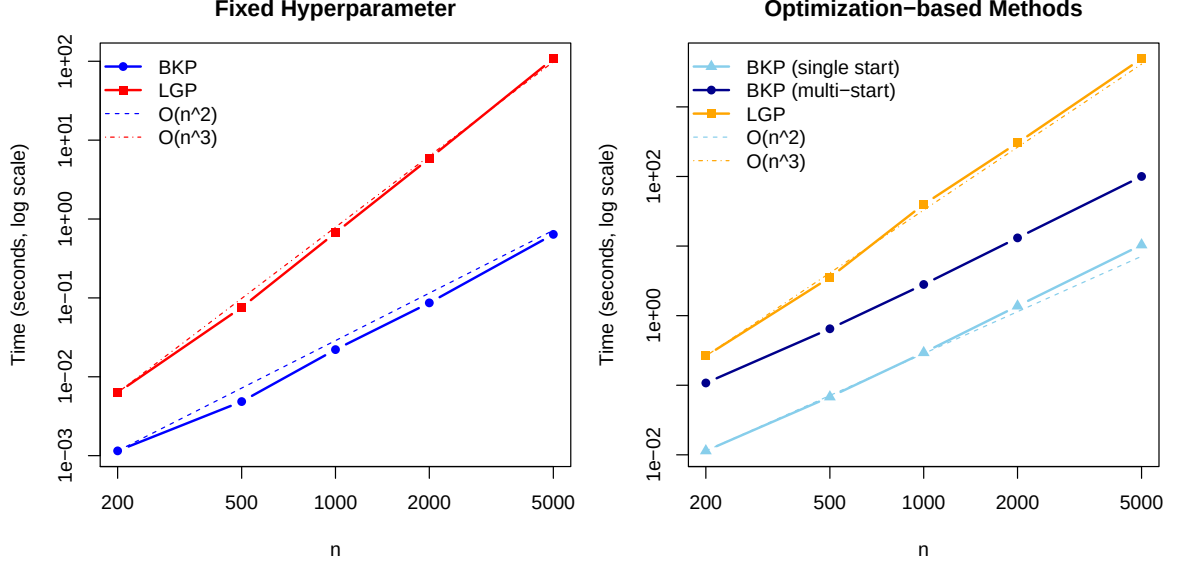


Figure 4: Comparison of computation times (in log scale) between BKP and LGP methods: (a) fixed hyperparameter; (b) optimization-based methods.

1. In the second setting, the kernel parameter is selected by LOOCV-based optimization, using either a single starting value `n_multi_start = 1` or the package default `n_multi_start = NULL`. Under the default rule, the number of starting values is $10d_\gamma$, where $d_\gamma = 1$ for isotropic kernels and $d_\gamma = d$ for anisotropic kernels. In the present benchmark, the default isotropic Gaussian kernel is used, so $d_\gamma = 1$ and the multi-start optimization uses 10 starting values.

The LGP model was implemented using the `gp_fit()` and `gp_optim()` functions from the **gplite** package (Piironen 2022). The **gplite** package uses additional starts only when optimization failure is detected; hence, its timing results are not directly comparable to a fixed multi-start optimization budget.

The timing experiments were conducted on the hardware described in the computational details. The reported computation times are averages over 20 independent repetitions to reduce variability due to random initialization and computational fluctuations.

The average computation times are shown in Figure 4. The observed trends are consistent with the theoretical scaling discussed in Remark 4: full BKP is a quadratic full-kernel method, whereas full LGP fitting involves cubic matrix-factorization operations. For BKP, the cost of multi-start optimization increases approximately in proportion to the number of starting values, although the exact ratio also depends on the number of loss evaluations taken by the optimizer from each start. In the present isotropic benchmark, the default multi-start setting uses $10d_\gamma = 10$ starts, so the multi-start timing is expected to be roughly one order of magnitude larger than the single-start timing, up to optimizer and implementation overhead. This illustrates the trade-off between increased optimization robustness and additional computation.

Example 4 We next consider a binary classification task using the *Two Spirals* dataset (Chalup and Wiklendt 2007), a well-known benchmark consisting of two intertwined spirals

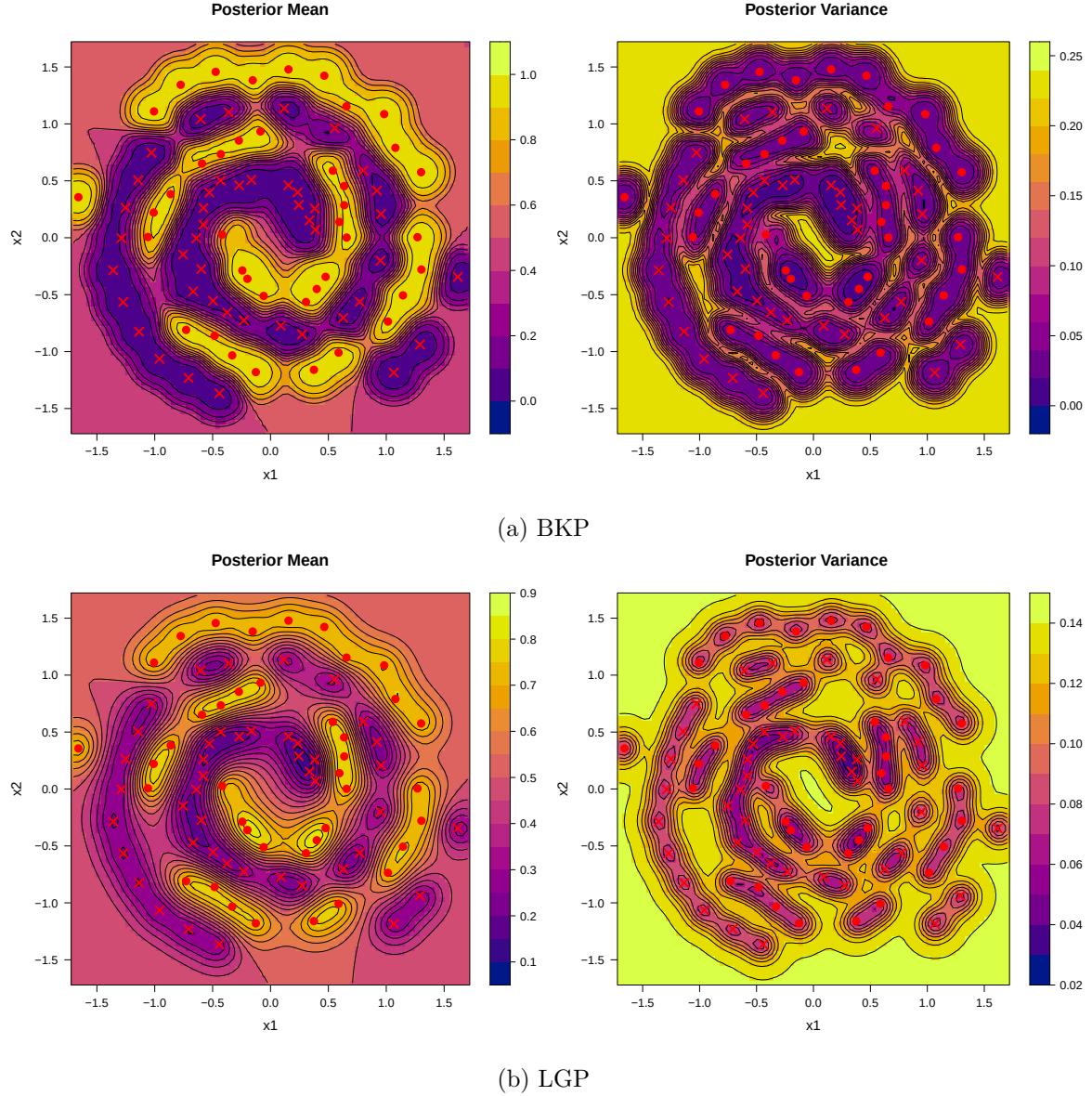


Figure 5: Probability surfaces for the Two Spirals example fitted by the BKP and LGP models. The left panels show the posterior mean class probabilities, and the right panels show the corresponding model-specific posterior variance summaries. Color shading represents the estimated class probability, while circles and crosses indicate training observations from the two classes.

in a bounded two-dimensional input space. This dataset is particularly challenging due to the complex, non-linearly separable class structure.

We generate $n = 120$ observations using the `mlbench.spirals()` function from the R package **mlbench** (Leisch and Dimitriadou 2024), with two complete rotations and additive Gaussian noise of standard deviation $\text{sd} = 0.05$. The inputs $\mathbf{x}$ are constrained to the domain $[-1.7, 1.7]^2$, and the binary class labels are encoded as 0 and 1. We fit the BKP model using

a fixed prior specification with $r_0 = 0.1$ and $p_0 = 0.5$.

```
R> n <- 120
R> data <- mlbench.spirals(n, cycles = 2, sd = 0.05)
R> train_indices <- sample(1:n, 0.7 * n)
R> X_train <- data$x[train_indices, ]
R> y_train <- as.numeric(data$classes[train_indices]) - 1 # Convert to 0/1 for BKP
R> X_test <- data$x[-train_indices, ]
R> y_test <- as.numeric(data$classes[-train_indices]) - 1
R> m <- rep(1, length(train_indices))
R> Xbounds <- rbind(c(-1.7, 1.7), c(-1.7, 1.7))
R> BKP_model_Class <- fit_BKP(
+   X_train, y_train, m, Xbounds = Xbounds,
+   prior = "fixed", r0 = 0.1, p0 = 0.5, loss = "log_loss")
R> prediction <- predict(BKP_model_Class, X_test)
```

Figure 5 presents the posterior mean and variance surfaces of the class probabilities. The upper-left panel shows that the BKP model captures the intricate spiral structure, with smoothly varying posterior mean probabilities that delineate the nonlinear decision boundary. The upper-right panel displays the corresponding posterior variance, which is generally larger near the boundary regions between the spirals. Circles and crosses indicate training observations from the two respective classes. These results illustrate the ability of BKP to model complex classification boundaries while providing model-based posterior uncertainty summaries.

We compare the prediction performance with the LGP model implemented by R package **gplite**.

```
R> gp <- gp_init(cf = cf_sexp(), lik = lik_bernoulli())
R> gp <- gp_optim(gp, X_train, y_train, verbose = FALSE)
R> prediction_gp <- gp_pred(gp, as.matrix(X_test), transform = TRUE)
```

The lower panels of Figure 5 show that the LGP fit produces a different probability surface and a more spatially diffuse variance pattern than BKP. Because the two methods use different posterior constructions, their variance surfaces should be interpreted as model-specific uncertainty summaries rather than as direct evidence of comparative calibration. The receiver operating characteristic (ROC) curves in Figure 6 provide a predictive comparison on the held-out observations: BKP achieves an AUC of 0.928, compared with 0.897 for LGP, indicating better discrimination for BKP on this particular train-test split.

4.2. DKP Model

This subsection presents illustrative examples based on the full DKP model. The scalable TwinDKP interface is illustrated together with TwinBKP in Section 4.3.

Example 5 Consider a one-dimensional three-class multinomial response setting. The input is defined on the interval $x \in [-2, 2]$, and the true class probability vector is given by

$$\pi(x) = \left[\frac{\pi_1(x)}{2}, \frac{\pi_2(x)}{2}, 1 - \frac{\pi_1(x)}{2} - \frac{\pi_2(x)}{2} \right]^\top,$$

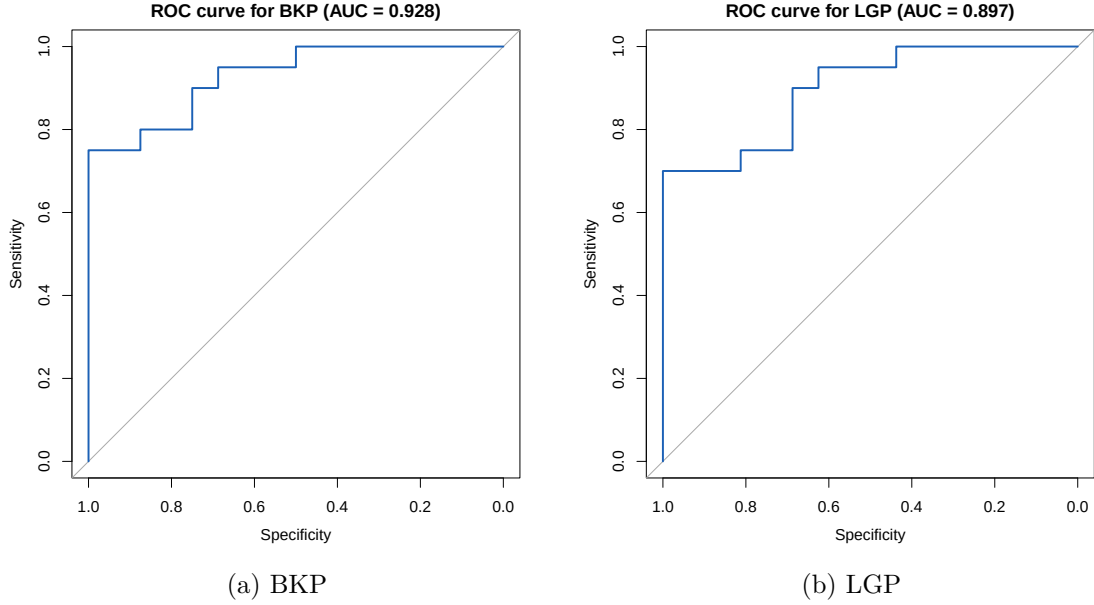


Figure 6: ROC curves and their respective AUCs for BKP and LGP models

where $\pi_1(x)$ and $\pi_2(x)$ are smooth functions defined in (16) and (17), respectively.

We generate $n = 30$ input locations using Latin hypercube sampling over the interval $[-2, 2]$. At each location, the response is a multinomial vector with probability $\pi(x)$ and a random total count sampled from $\{1, \dots, 150\}$. The DKP model is then fitted using the `fit_DKP()` function and visualized with the `plot()` method:

```
R> n <- 30
R> Xbounds <- matrix(c(-2, 2), nrow = 1)
R> X <- lhs(n = n, rect = Xbounds)
R> true_pi <- true_pi_fun(X)
R> m <- sample(150, n, replace = TRUE)
R> Y <- t(sapply(1:n, function(i) rmultinom(1, size = m[i], prob = true_pi[i, ])))
R> DKP_model_1D <- fit_DKP(X, Y, Xbounds = Xbounds)
R> plot(DKP_model_1D)
```

Figure 7 shows that the DKP fitted mean curves follow the main features of the true class-probability functions, while the shaded bands summarize posterior uncertainty. The predictive mean curves align with the true underlying structure, and the shaded bands reflect posterior uncertainty.

Example 6 Consider a two-dimensional three-class multinomial response setting. Let $\mathbf{x} = [x_1, x_2]^\top \in [0, 1]^2$, and define the true class probability function as

$$\pi(\mathbf{x}) = \left[\frac{\pi_3(\mathbf{x})}{2}, \frac{\pi_4(\mathbf{x})}{2}, 1 - \frac{\pi_3(\mathbf{x})}{2} - \frac{\pi_4(\mathbf{x})}{2} \right]^\top,$$

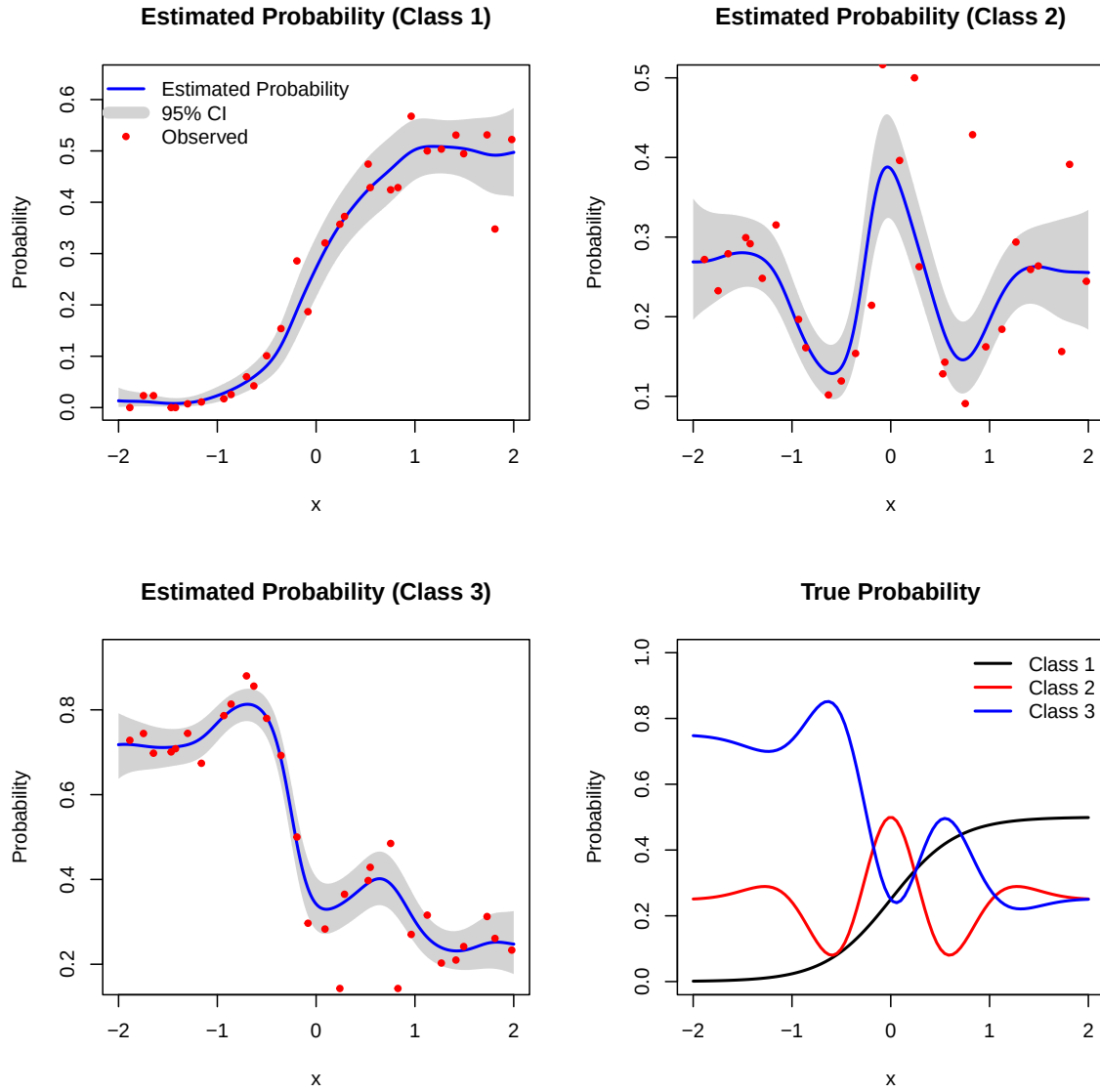


Figure 7: Posterior inference from the fitted DKP model for a one-dimensional three-class problem

where $\pi_3(\mathbf{x})$ is defined in (18), and

$$\pi_4(\mathbf{x}) = \sin(\pi x_1) \sin(\pi x_2).$$

We generate $n = 100$ input points using Latin hypercube sampling over $[0, 1]^2$. At each location, the response is a multinomial vector with probability $\boldsymbol{\pi}(\mathbf{x})$ and a total count randomly sampled from $\{1, \dots, 150\}$. We fit the DKP model using `fit_DKP()` and visualize the results with the `plot()` method:

```
R> n <- 100
R> Xbounds <- matrix(c(0, 0, 1, 1), nrow = 2)
```

```

R> X <- lhs(n = n, rect = Xbounds)
R> true_pi <- true_pi_fun(X)
R> m <- sample(150, n, replace = TRUE)
R> Y <- t(sapply(1:n, function(i) rmultinom(1, size = m[i], prob = true_pi[i, ])))
R> DKP_model_2D <- fit_DKP(X, Y, Xbounds = Xbounds)

```

Figures 8–10 display the ground truth surfaces and corresponding posterior inference for each of the three classes. In each figure, the top panel shows the true class probability surface, while the bottom row presents the posterior mean, variance, and uncertainty bounds produced by the DKP model.

Example 7 Consider another three-class classification task based on the well-known *Iris* dataset available in R. This classic benchmark dataset contains measurements of three iris species—*setosa*, *versicolor*, and *virginica*—each described by four features: sepal length, sepal width, petal length, and petal width. Due to the overlap in feature space, particularly between *versicolor* and *virginica*, the class boundaries are not linearly separable, making this dataset a standard testbed for evaluating multi-class classification algorithms. For visualization purposes, we restrict our analysis to the first two features: sepal length and sepal width. We fit the DKP model using a fixed prior specification with $r_0 = 0.01$ and $p_0 = \text{rep}(1/3, 3)$, assuming equal prior probability for each class.

```

R> data(iris)
R> X <- as.matrix(iris[, 1:2])
R> Xbounds <- rbind(c(4.2, 8), c(1.9, 4.5))
R> labels <- iris$Species
R> Y <- model.matrix(~ labels - 1) # expand factors to a set of dummy variables
R> train_indices <- sample(1:nrow(iris), 0.7 * nrow(iris))
R> X_train <- X[train_indices, ]
R> Y_train <- Y[train_indices, ]
R> DKP_model_Class <- fit_DKP(
+   X_train, Y_train, Xbounds = Xbounds, loss = "log_loss",
+   prior = "fixed", r0 = 0.01, p0 = rep(1/3, 3))
R> X_test <- X[-train_indices, ]
R> Y_test <- Y[-train_indices, ]
R> dkp_pred_probs <- predict(DKP_model_Class, X_test)$mean

```

Figure 11 provides a visual comparison of the fitted DKP and multiclass LGP models. The upper-left panel shows that the DKP model clearly separates *setosa* from the other two species, while producing a smooth transition between the overlapping *versicolor* and *virginica* regions. The upper-right panel displays the corresponding maximum posterior class probability $\max_j \pi_j(\mathbf{x})$. Values near 1 indicate decisive class assignments, whereas values closer to $1/3$ indicate greater ambiguity among the three classes. The lower-confidence regions are concentrated primarily near the fitted class boundaries, yielding a coherent representation of classification confidence.

For comparison, we fit a multiclass Gaussian-process classifier using the **kernlab** package (Karatzoglou *et al.* 2004). The `gausspr()` implementation fits pairwise binary Gaussian-process classifiers and combines their probability estimates through pairwise coupling. For

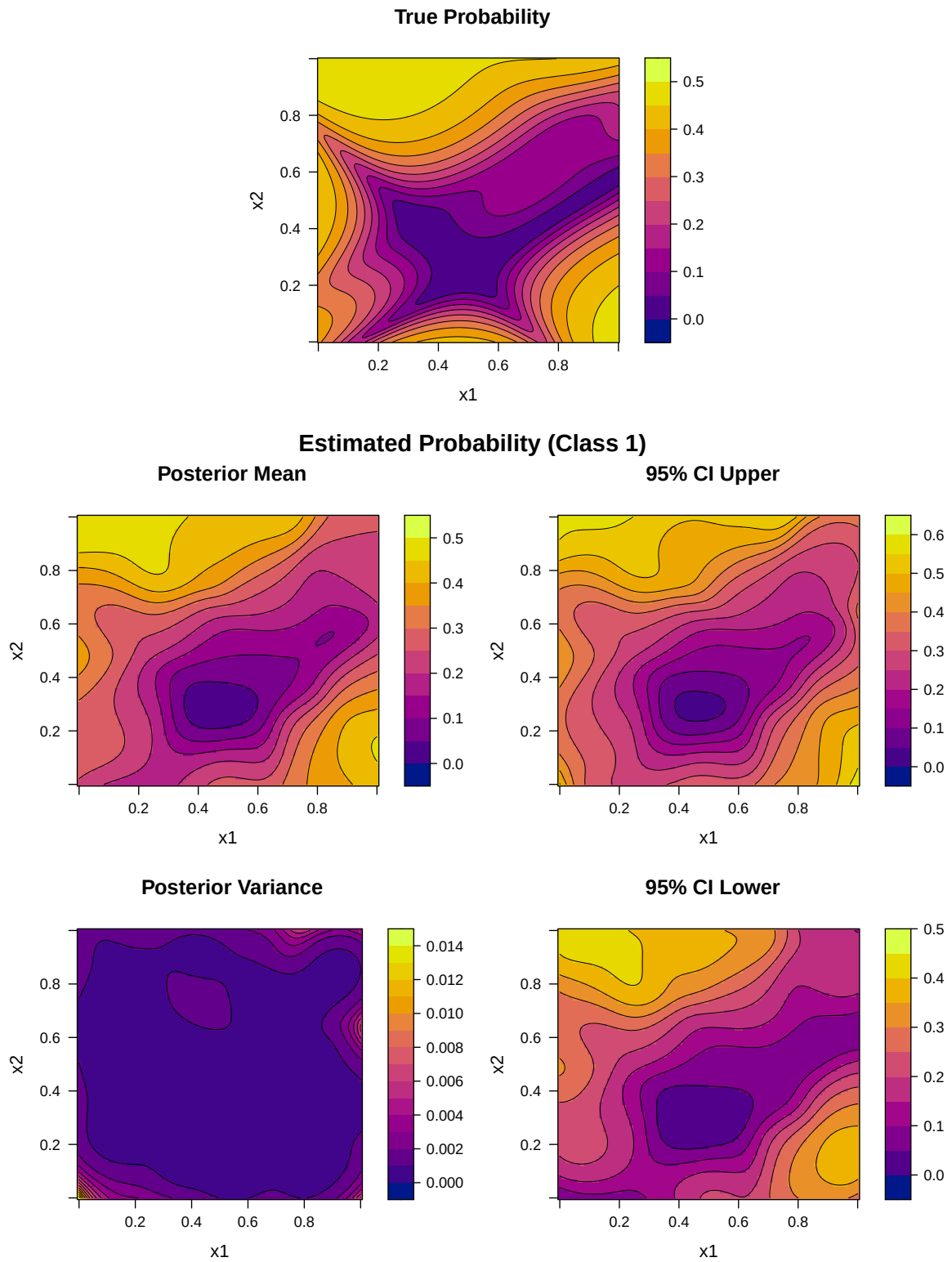


Figure 8: Class 1: true distribution (top) and DKP model posterior summaries (bottom)

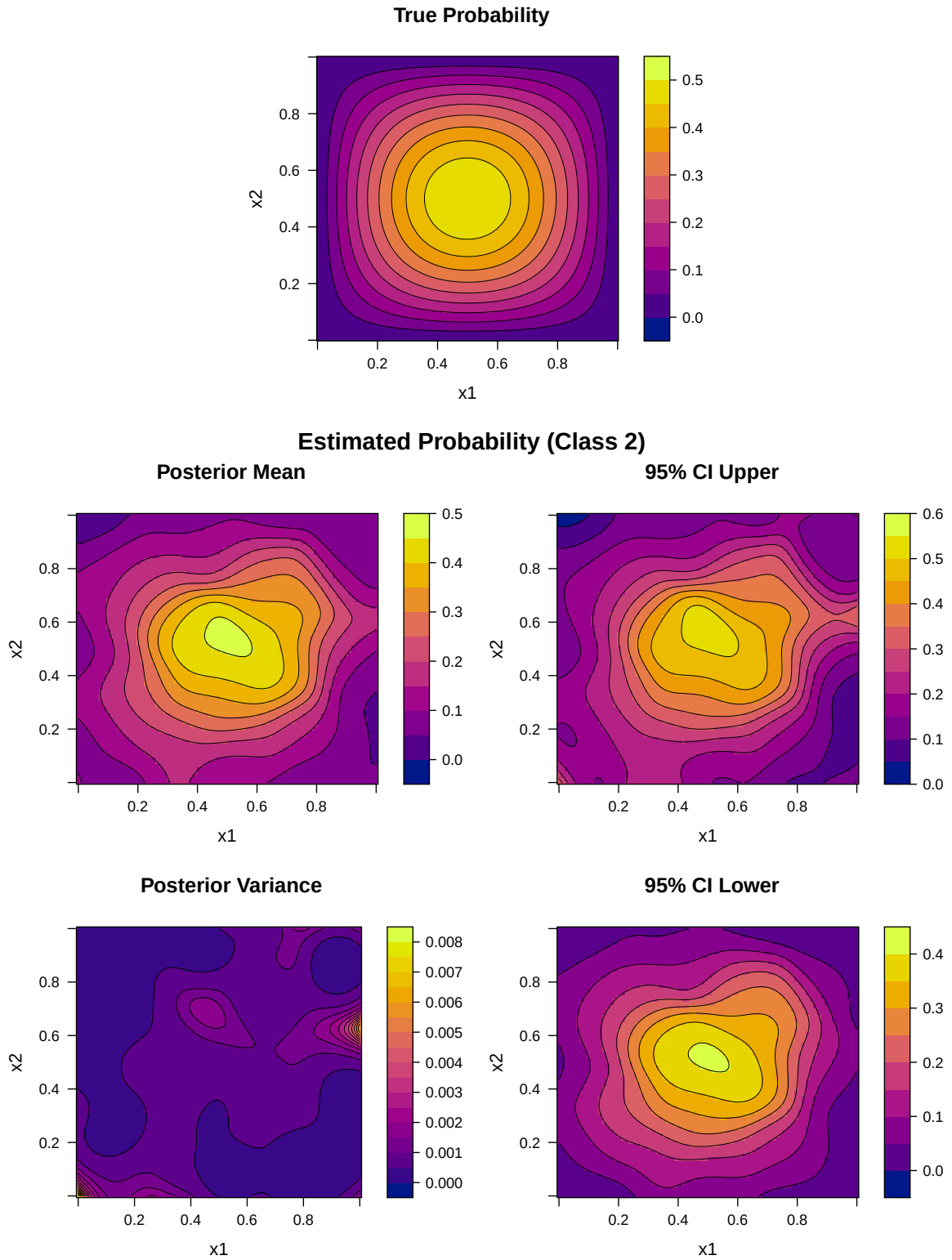


Figure 9: Class 2: true distribution (top) and DKP model posterior summaries (bottom)

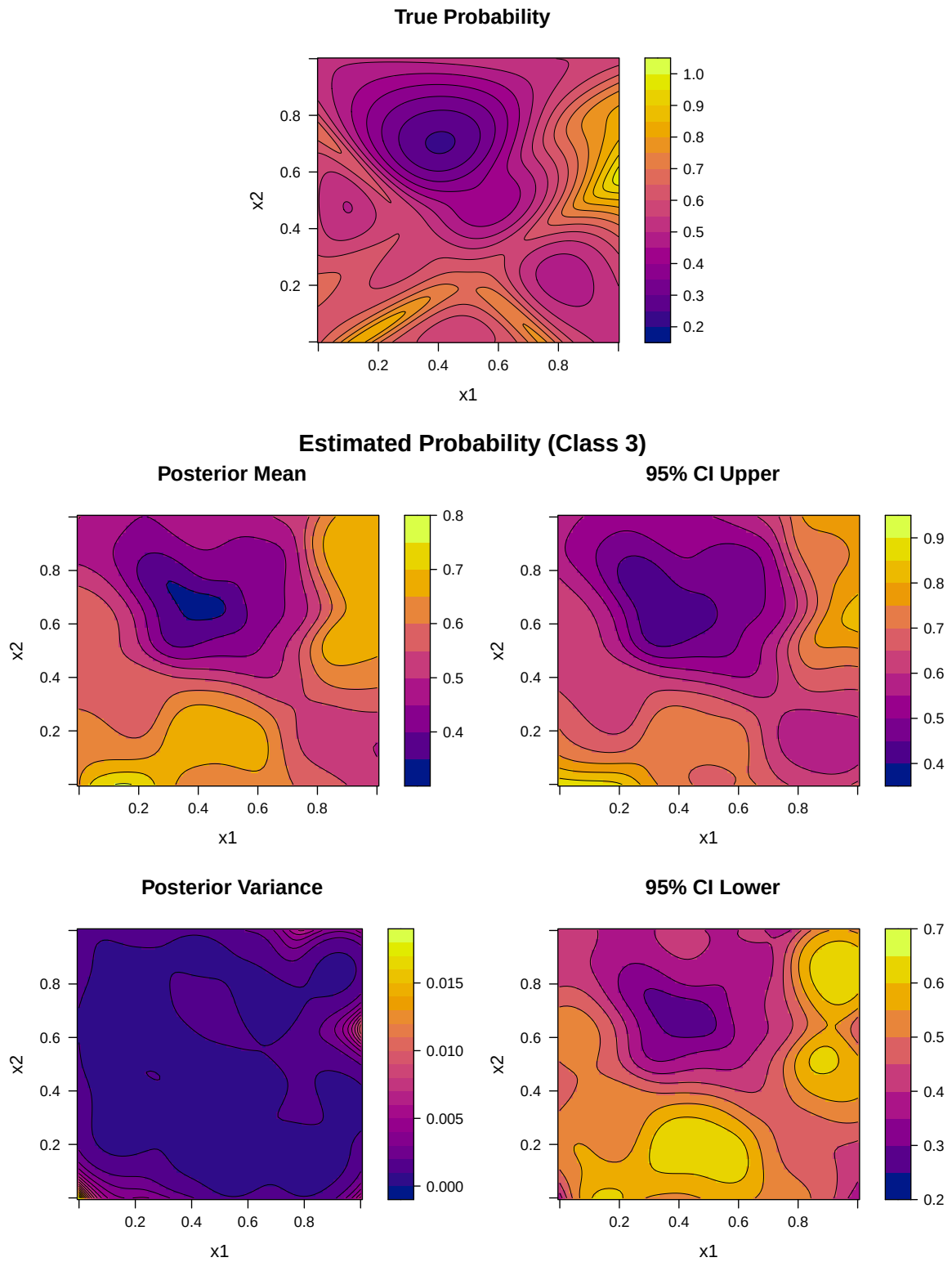


Figure 10: Class 3: true distribution (top) and DKP model posterior summaries (bottom)

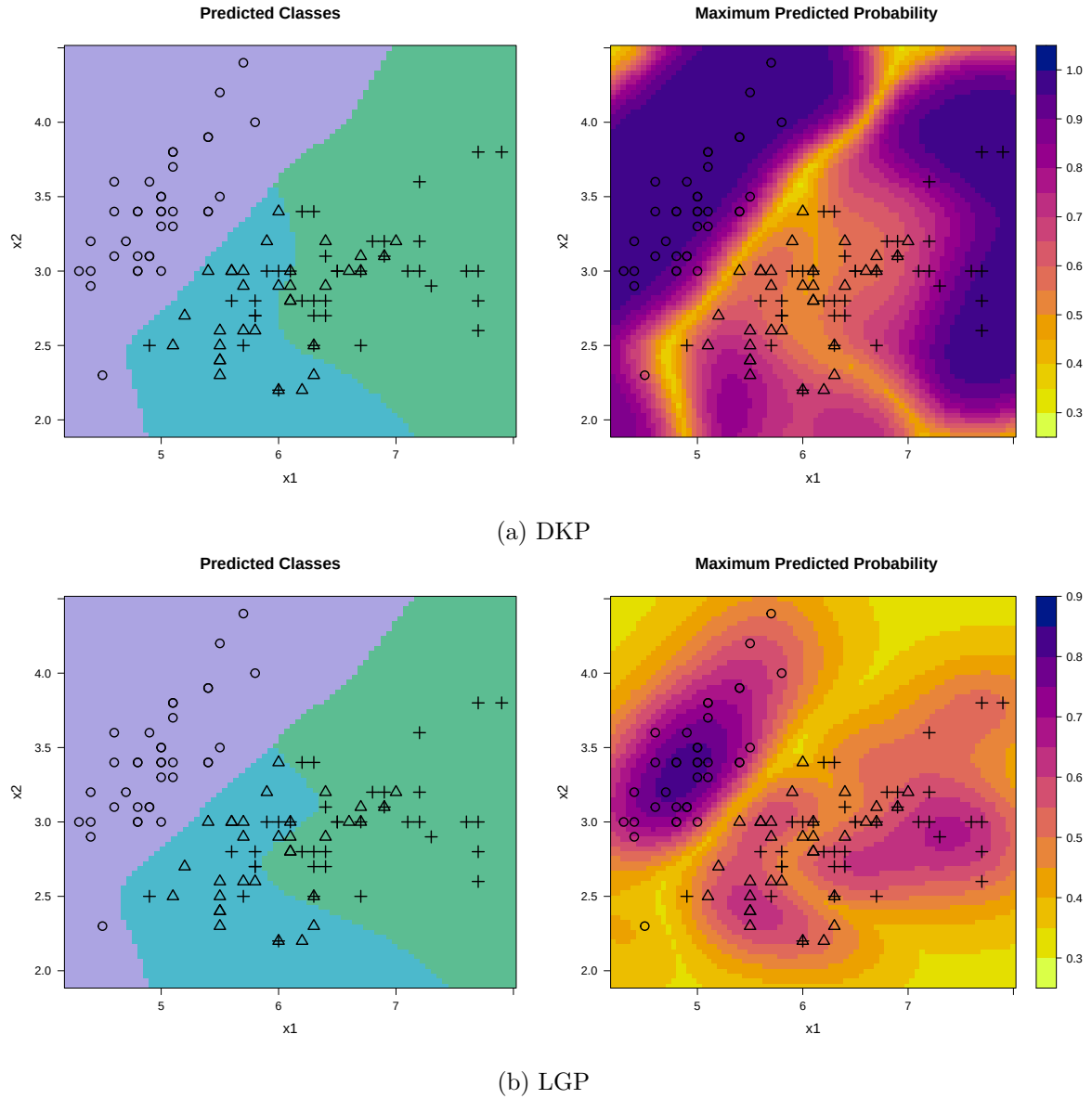


Figure 11: Classification and confidence visualization for the DKP and multiclass LGP fits on the *Iris* dataset. The left panels show the predicted classification regions, with the training observations overlaid using class-specific symbols. The right panels show the maximum predicted class probability as a model-specific summary of classification confidence. Lower values indicate less decisive class-probability assignments, which typically occur near class boundaries.

evaluation, we compute one-vs-rest ROC curves for the three classes and report the corresponding macro-average AUC.

```
R> iris_data <- data.frame(
+   Sepal.Length = iris$Sepal.Length,
+   Sepal.Width = iris$Sepal.Width,
```

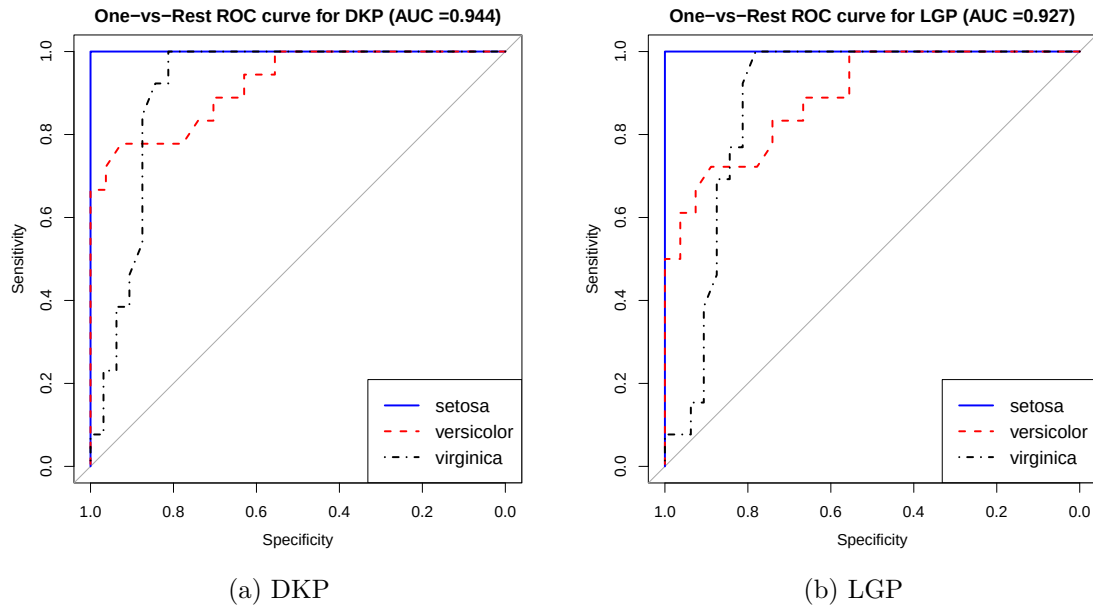


Figure 12: One-vs-Rest ROC curves and their respective macro-average AUCs for DKP and LGP models

```

+   Species = iris$Species
+ )
R> iris_train <- iris_data[train_indices, ]
R> iris_test <- iris_data[-train_indices, ]
R> gausspr_model <- gausspr(Species ~ ., data = iris_train,
+                           kernel = "rbfdot", kpar = "automatic")
R> lgp_pred_probs <- predict(gausspr_model, newdata = iris_test,
+                           type = "probabilities")

```

Compared with the DKP fit, the LGP model produces less smooth and more fragmented decision regions, particularly between the clusters corresponding to *versicolor* and *virginica*, as shown in the lower-left panel of Figure 11. Its maximum predicted probability surface in the lower-right panel is also less spatially coherent, with isolated pockets of relatively high or low confidence appearing within otherwise homogeneous class regions. In contrast, the DKP confidence surface varies more systematically with the fitted decision boundaries and provides a more stable model-based representation of classification confidence. The corresponding ROC curves in Figure 12 yield a macro-average AUC of 0.944 for DKP and 0.927 for LGP. Taken together, the smoother decision structure, more coherent confidence surface, and higher macro-average AUC indicate that DKP provides better predictive discrimination and a more stable representation of classification confidence than LGP in this example.

4.3. TwinBKP and TwinDKP Models

This subsection illustrates the TwinBKP and TwinDKP interfaces. The examples reuse the settings from Examples 2 and 5, but increase the sample size and replace the full-kernel fitting functions by their global-local counterparts. The purpose is to show that the scalable variants

retain the same response formats and downstream S3 workflow, while replacing full kernel aggregation with twinning-selected global subsets and prediction-specific local neighbours.

Example 8 We first revisit the nonlinear binomial response setting in Example 2. The true probability function in (17) is unchanged, but the number of input locations is increased from 30 to 500. The full BKP call is then replaced by `fit_TwinBKP()`, which uses a twinning-selected global subset together with prediction-specific local neighbours.

```
R> n <- 500
R> Xbounds <- matrix(c(-2, 2), nrow = 1)
R> X <- lhs(n = n, rect = Xbounds)
R> true_pi <- true_pi_fun(X)
R> m <- sample(100, n, replace = TRUE)
R> y <- rbinom(n, size = m, prob = true_pi)
R> TwinBKP_model_1D_2 <- fit_TwinBKP(X, y, m, Xbounds = Xbounds)
R> print(TwinBKP_model_1D_2)
R> plot(TwinBKP_model_1D_2)
```

Twin Beta Kernel Process (TwinBKP) Model

```
Number of observations (n): 500
Input dimension (d):      1
Global kernel:            gaussian
Local kernel:             wendland
Isotropic:               TRUE
theta_g:                 0.05
theta_l:                 0.0545
Loss function:           brier
Loss minimum:            0.03694
Prior:                   noninformative
Global subset size (g):   22 (target 22)
Local neighbours (l):    25
Twins runs:              5
```

The first three arguments have the same meaning as in `fit_BKP()`: `X` is the input matrix, `y` contains the observed success counts, and `m` contains the binomial trial sizes. The additional global-local structure is handled internally. In this run, the default rule selects 22 global points and 25 local neighbours per prediction location. The fitted ‘TwinBKP’ object supports the same downstream workflow as a full ‘BKP’ object.

Figure 13 shows that the TwinBKP fit follows the main nonlinear pattern of the probability function while using a compact global subset and local nearest-neighbour refinements. The posterior summaries retain the beta-binomial interpretation of the full BKP model, but the pseudo-count aggregation is performed over the union of the global subset and the prediction-specific local subset rather than over all training observations.

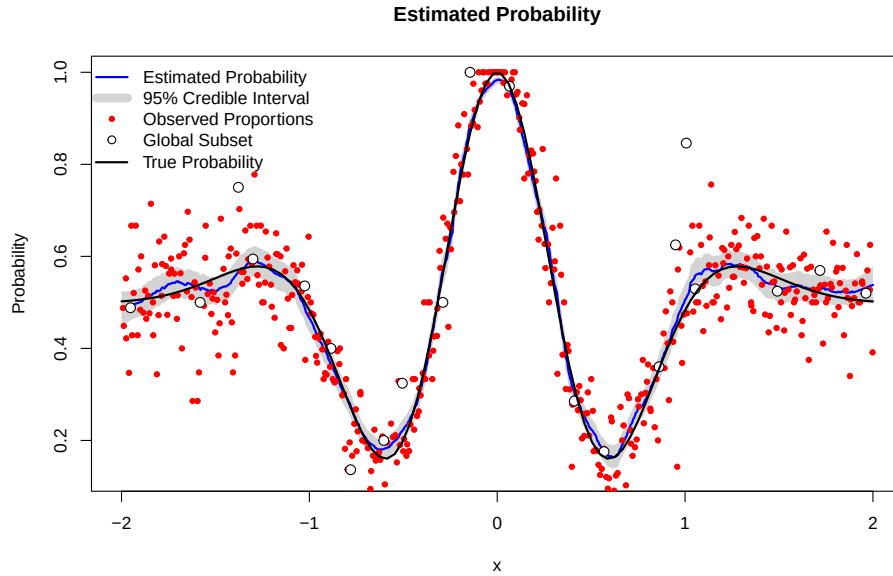


Figure 13: Posterior inference from the fitted TwinBKP model for the one-dimensional non-linear binomial example. The blue curve is the posterior mean, the gray band is the pointwise 95% credible interval, the red points are observed proportions, and the open circles indicate the twinning-selected global subset.

Example 9 We next revisit the one-dimensional three-class multinomial response setting in Example 5 to illustrate the TwinDKP interface. The class-probability functions are unchanged, but the number of input locations is increased from 30 to 500. The full DKP call is replaced by `fit_TwinDKP()`. As in the full DKP model, the response is supplied as an `n` by `q` matrix of multinomial counts.

```
R> n <- 500
R> Xbounds <- matrix(c(-2, 2), nrow = 1)
R> X <- lhs(n = n, rect = Xbounds)
R> true_pi <- true_pi_fun(X)
R> m <- sample(150, n, replace = TRUE)
R> Y <- t(sapply(1:n, function(i) rmultinom(1, size = m[i], prob = true_pi[i, ])))
R> TwinDKP_model_1D <- fit_TwinDKP(X, Y, Xbounds = Xbounds)
R> plot(TwinDKP_model_1D)
```

The fitted ‘TwinDKP’ object can be visualized using the same `plot()` method as a full ‘DKP’ object. Figure 14 reports the fitted TwinDKP posterior summaries, with the true class-probability curves added as a fourth panel for comparison.

The TwinDKP fit follows the main features of the three class-probability functions while using the same global-local approximation as TwinBKP. The posterior summaries retain the Dirichlet-multinomial interpretation of the full DKP model, but the pseudo-count vector at each prediction location is aggregated over the union of the twinning-selected global subset and the prediction-specific local subset.

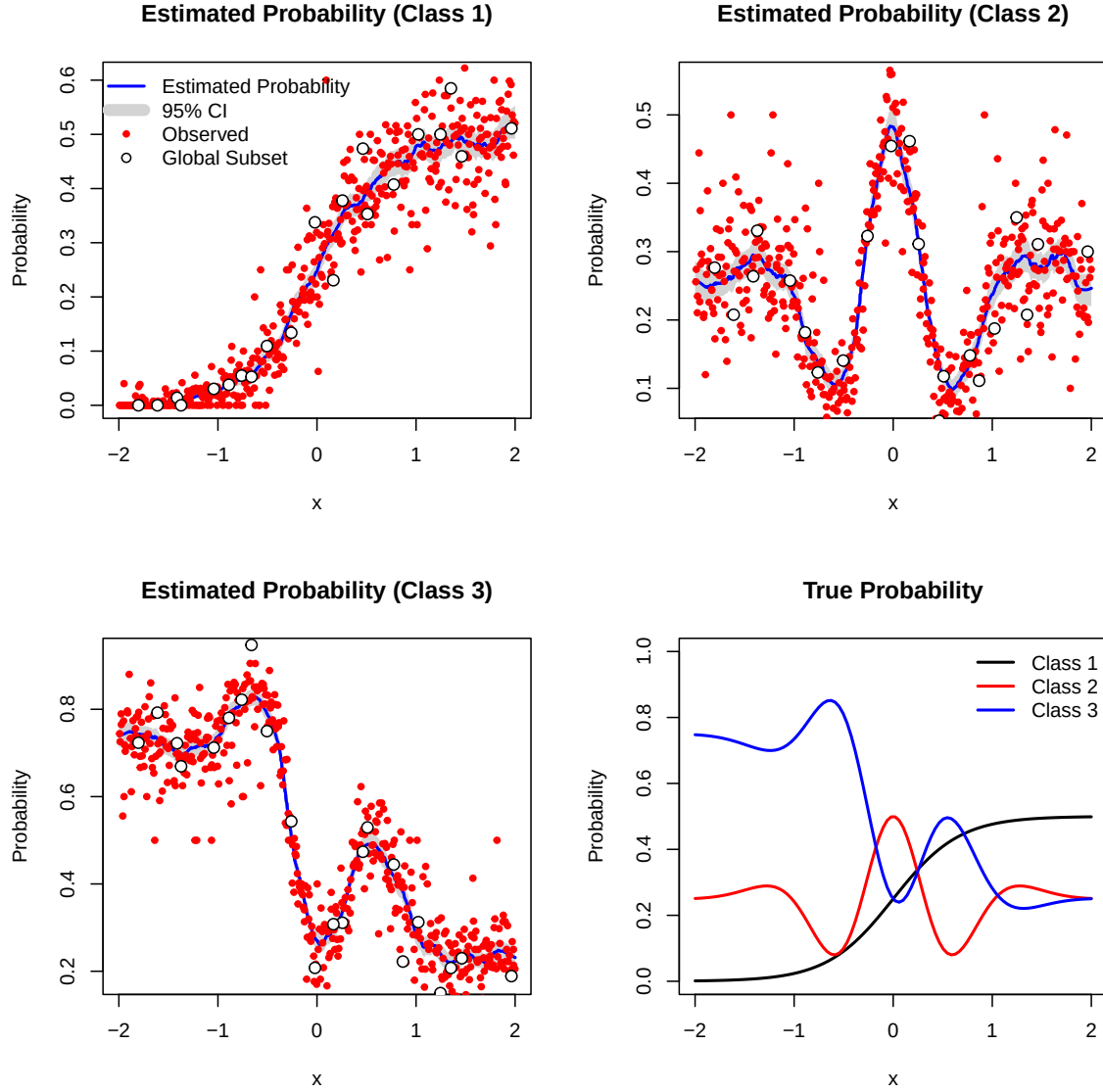


Figure 14: Posterior inference from the fitted TwinDKP model for the one-dimensional three-class multinomial example. The first three panels show the posterior mean class probabilities, pointwise 95% credible intervals, observed class proportions, and the twinning-selected global subset. The fourth panel shows the true class-probability functions.

5. Real-data Applications

This section illustrates the use of **BKP** in two real-data applications: spatial prevalence mapping of *Loa loa* infection using aggregated binomial counts, and species distribution modeling for the Mourning Warbler using binary presence–absence data and high-dimensional environmental covariates. The data sources and preprocessing procedures are described in the corresponding subsections.

5.1. Loa loa infection prevalence mapping

Data Preparation

The `loaloa` dataset, available in the **RiskMap** package (Giorgi 2025), contains observations from 197 survey locations in North Cameroon collected between 1991 and 2001 (Diggle *et al.* 2007). For each location, the dataset records the geographical coordinates (`LONGITUDE` and `LATITUDE`), the number of infected individuals (`NO_INF`), the total number examined (`NO_EXAM`), and additional environmental variables, including altitude and the maximum normalized difference vegetation index (`NDVI`).

In this analysis, geographical coordinates are used as the input variables $\mathbf{X}$, the number of infected individuals as the binomial success count y , and the number examined as the trial count m . For illustration, the locations are randomly divided into a training set containing 70% of the observations and a testing set containing the remaining 30%.

```
R> data("loaloa", package = "RiskMap")
R> # Extract coordinates, infected counts, and numbers examined
R> X <- as.matrix(loaloa[, c("LONGITUDE", "LATITUDE")])
R> y <- loaloa$NO_INF
R> m <- loaloa$NO_EXAM
R> # Randomly split into training (70%) and testing (30%) sets
R> train_idx <- sample(1:nrow(loaloa), 0.7 * nrow(loaloa))
R> X_train <- X[train_idx, ]
R> y_train <- y[train_idx]
R> m_train <- m[train_idx]
R> X_test <- X[-train_idx, ]
R> y_test <- y[-train_idx]
R> m_test <- m[-train_idx]
```

Figure 15 displays the observed infection proportion y/m at each survey location. Point size represents the number examined and therefore indicates the nominal amount of binomial information available at each location, while color represents the observed infection proportion. Point shape distinguishes the training and testing locations. The observed proportions exhibit substantial spatial heterogeneity, including groups of locations with elevated prevalence between approximately 10°E and 12°E, and another group near 14°E–15°E.

Model Fitting and Visualization

We fit a BKP model to the training data using `fit_BKP()`. The default specification uses the Brier score for LOOCV-based length-scale selection, a non-informative beta prior, and an isotropic Gaussian kernel. The bounds supplied through `Xbounds` rescale longitude and latitude to the unit square before fitting.

```
R> Xbounds <- matrix(c(7.8, 15.3, 3.1, 7.0), ncol = 2, byrow = TRUE)
R> bkp_fit <- fit_BKP(X_train, y_train, m_train, Xbounds = Xbounds)
R> summary(bkp_fit)
```

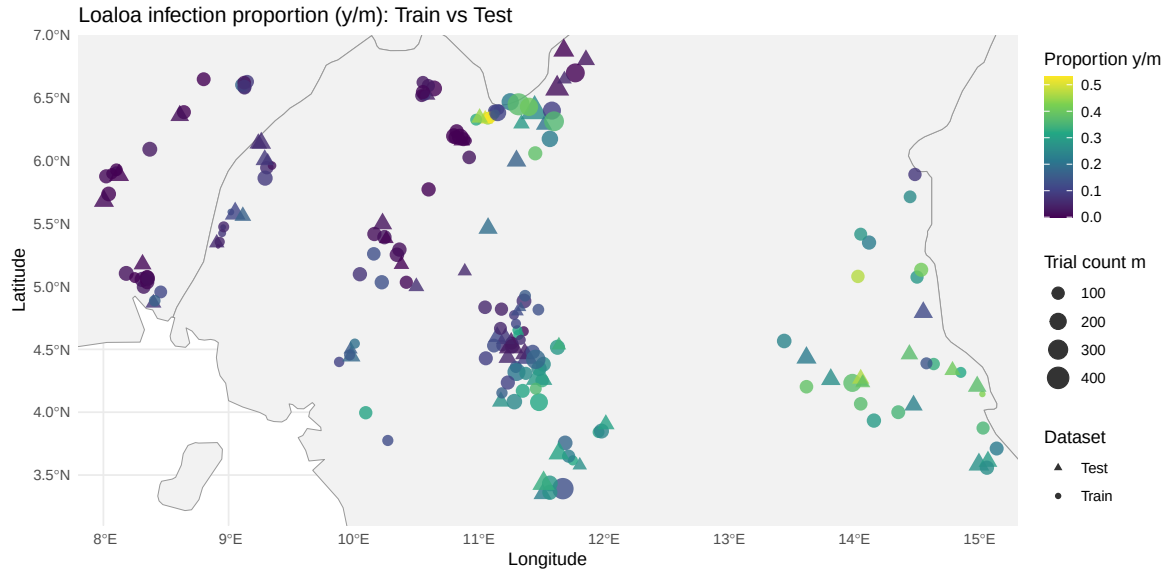


Figure 15: Observed *Loa loa* infection proportions y/m across North Cameroon. Point size is proportional to the number examined, color represents the observed infection proportion, and shape distinguishes the randomly selected training locations (circles; 70%) from the testing locations (triangles; 30%).

Beta Kernel Process (BKP) Model

```

Number of observations (n): 137
Input dimensionality (d): 2
Kernel type: (isotropic) gaussian
Optimized kernel parameters: 0.0486
Minimum achieved loss: 0.00943
Loss function: brier
Prior type: noninformative

```

```

Posterior predictive summary (training points):
              Mean Median   SD   Min   Max
Posterior means 0.1509 0.1273 0.1059 0.0077 0.4134
Posterior variances 0.0003 0.0001 0.0004 0.0000 0.0026

```

For comparison, we also fit a logistic Gaussian process (LGP) model using `gp_init()` and `gp_optim()` from the **gplite** package (Pironen 2022).

```

R> gp <- gp_init(cf = cf_sexp(), lik = lik_binomial())
R> gp <- gp_optim(gp, X_train, y_train, trials = m_train, verbose = FALSE)

```

Figure 16 compares the fitted probability and variance surfaces from the two models. The BKP probability surface contains several spatially connected regions of elevated predicted

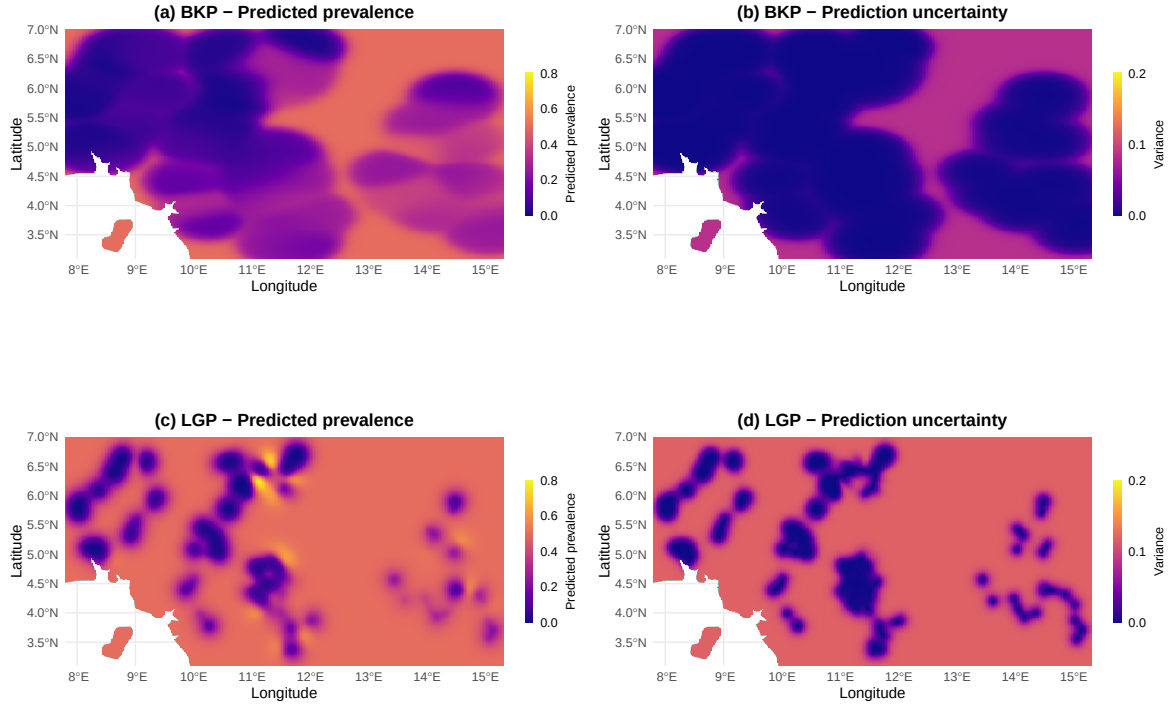


Figure 16: Probability-scale prediction summaries for the BKP and LGP models fitted to the *Loa loa* data. The left column shows fitted infection probabilities, and the right column shows the corresponding model-specific variance surfaces. Common color limits are used for the two probability panels and, separately, for the two variance panels.

prevalence, including regions corresponding broadly to the higher observed proportions in Figure 15. The LGP surface is more diffuse over parts of the study region and contains a larger number of small-scale local features. These differences reflect the distinct smoothing constructions of the two methods: BKP aggregates kernel-weighted success and failure pseudo-counts directly on the probability scale, whereas LGP performs inference through a latent Gaussian process and a nonlinear link function.

The variance surfaces also exhibit different spatial patterns. For BKP, larger posterior variances are concentrated mainly around transitions between lower- and higher-prevalence regions and in locations with weaker nearby data support. The LGP variance is more broadly distributed over the study region, with local reductions around some observed locations. Because BKP and LGP construct probability-scale uncertainty differently, these variance maps should be interpreted as model-specific uncertainty summaries rather than as directly comparable frequentist coverage measures. A separate simulation study of empirical pointwise interval coverage is reported in Appendix A.

Predictive Performance on the Test Data

We next evaluate the fitted probabilities at the randomly withheld testing locations using the

Brier score.

```
R> all_preds <- list()
R> all_preds$BKP <- predict(bkp_fit, Xnew = X_test)$mean
R> all_preds$LGP <- gp_pred(gp, X_test, transform = TRUE)$mean
R> # Calculate the empirical success rate for the test data
R> pi_test <- y_test / m_test
R> # Calculate the Mean Squared Error (Brier Score)
R> mse_bkp <- mean((all_preds$BKP - pi_test)^2)
R> mse_gp <- mean((all_preds$LGP - pi_test)^2)
```

For this random train–test split, the BKP model achieves a Brier score of 0.0073, compared with 0.0188 for the LGP model. The lower score indicates that the BKP predictions agree more closely, on average, with the observed infection proportions at the withheld locations. Together with the spatial patterns in Figure 16, these results show that BKP provides an accurate and spatially coherent fit for this *Loa loa* prevalence-mapping application.

5.2. Mourning Warbler distribution modeling

We next illustrate the use of BKP and TwinBKP for species distribution modeling using presence–absence observations of the Mourning Warbler in North America. Unlike the previous application, the models are fitted in an eight-dimensional environmental covariate space rather than directly in geographic coordinates. The example compares BKP, TwinBKP, and LGP using a predefined spatially structured test set and subsequently projects the fitted environmental relationships back into geographic space.

Data Preparation

The occurrence data are derived from the 2011 North American Breeding Bird Survey (BBS) (Sauer *et al.* 2014), using a compiled subset provided by Harris (2015) and subsequently analyzed by Golding and Purse (2016). The dataset contains geographic coordinates, a binary response y indicating presence ($y = 1$) or absence ($y = 0$), and eight continuous bioclimatic covariates.

Following Golding and Purse (2016), longitude and latitude are used only to define the spatial partition and visualize the results; they are not included as model predictors. Instead, all three models use the same eight bioclimatic variables. The predefined disc-based spatial partition contains 1,613 training observations and 301 testing observations, with the testing observations forming geographically contiguous withheld regions. This partition provides a more demanding assessment of spatial transfer than a conventional random train–test split.

```
R> # Load observation data and define training/testing splits
R> data <- read.csv("code/data/Mourning_Warbler.csv", stringsAsFactors = FALSE)
R> train_idx <- which(data$split == "training")
R> test_idx <- which(data$split == "testing")
R> bio_cols <- grep("^wc2", names(data), value = TRUE)
R> # Extract 8 bioclimatic covariates
R> X_all <- as.matrix(data[, bio_cols])
```

Distribution data – Mourning Warbler

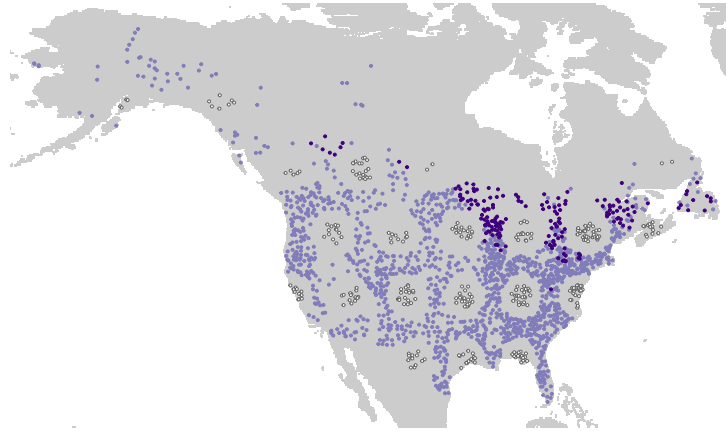


Figure 17: Spatial distribution of the Mourning Warbler observations. Dark purple points indicate training presences, light purple points indicate training absences, and white points indicate the testing observations withheld under the disc-based spatial partition.

```
R> X_train <- X_all[train_idx, ]; y_train <- data$y[train_idx]
R> m_train <- rep(1L, length(y_train))
R> X_test  <- X_all[test_idx, ]; y_test  <- data$y[test_idx]
R> Xbounds <- t(apply(X_train, 2, range))
R> # Load and crop the environmental raster for spatial projection
R> library(terra)
R> tif_files <- file.path("code/data/climate/wc2.1_10m", paste0(bio_cols, ".tif"))
R> na_clim <- crop(rast(tif_files), ext(-170.15, -46.70, 22.23, 71.90))
R> names(na_clim) <- bio_cols
```

Figure 17 shows the spatial distribution of the training and testing observations. The withheld observations occur in geographically contiguous regions created by the disc-based partition, rather than being randomly interspersed among the training locations.

Model Fitting and Predictive Evaluation

We fit the full BKP model using `fit_BKP()` with a fixed prior precision $r_0 = 0.1$, the log-loss criterion, and an isotropic Gaussian kernel. Because `p0` is not supplied, the prior mean is set to the empirical presence proportion in the training data. We then fit `TwinBKP` using the same global kernel and prior specification, together with the default compactly supported Wendland kernel for the local update. For comparison, an LGP model with a Bernoulli likelihood and squared-exponential covariance function is fitted using the **gplite** package.

The elapsed times reported below include model fitting, hyperparameter optimization, and probability prediction at the 301 testing locations.

```
R> # 1. Fit full BKP
R> bkp_fit <- fit_BKP(X_train, y_train, m_train, Xbounds = Xbounds,
```

Model	AUC	Brier score	Elapsed time (s)
BKP	0.888	0.0691	9.04
TwinBKP	0.913	0.0674	0.05
LGP	0.852	0.0798	541.82

Table 3: Predictive performance and elapsed computation time for the Mourning Warbler application. AUC and Brier scores are evaluated on the predefined spatial test set containing 301 observations. Elapsed time includes model fitting, hyperparameter optimization, and prediction at the testing locations and is reported for a single replication run on the hardware described in the computational details.

```

+                               prior = "fixed", r0 = 0.1, loss = "log_loss")
R> bkp_pred <- predict(bkp_fit, Xnew = X_test)
R>
R> # 2. Fit TwinBKP
R> twin_fit <- fit_TwinBKP(X_train, y_train, m_train, Xbounds = Xbounds,
+                          prior = "fixed", r0 = 0.1, loss = "log_loss")
R> twin_pred <- predict(twin_fit, Xnew = X_test)
R>
R> # 3. Fit LGP (via gplite)
R> gp <- gp_init(cf = cf_sexp(), lik = lik_bernoulli())
R> gp <- gp_optim(gp, X_train, y_train, verbose = FALSE)
R> gp_pred <- gp_pred(gp, X_test, transform = TRUE)

```

Predictive performance on the spatially withheld testing set is summarized using the area under the receiver operating characteristic curve (AUC) and the Brier score. Table 3 reports the predictive performance and elapsed computation times for the three fitted models.

On this predefined spatial test set, TwinBKP achieves the highest AUC of 0.913 and the lowest Brier score of 0.0674. The full BKP model obtains an AUC of 0.888 and a Brier score of 0.0691, while the corresponding LGP values are 0.852 and 0.0798. Thus, both BKP-based fits provide better discrimination and smaller probability errors than the fitted LGP model in this application.

TwinBKP also has the shortest observed elapsed time at 0.05 seconds, compared with 9.04 seconds for full BKP and 541.82 seconds for LGP. These results illustrate the computational benefit of replacing full-data kernel aggregation with the twinning-based global-local approximation in this eight-dimensional example.

Spatial Projection

To visualize the fitted environmental relationships geographically, each model is evaluated at raster cells having complete values for all eight bioclimatic covariates. The resulting occurrence probabilities and variance summaries are then mapped to the corresponding geographic locations. Because longitude and latitude are not model predictors, these maps represent geographic projections of relationships learned in environmental covariate space rather than direct spatial smoothing of the occurrence data.

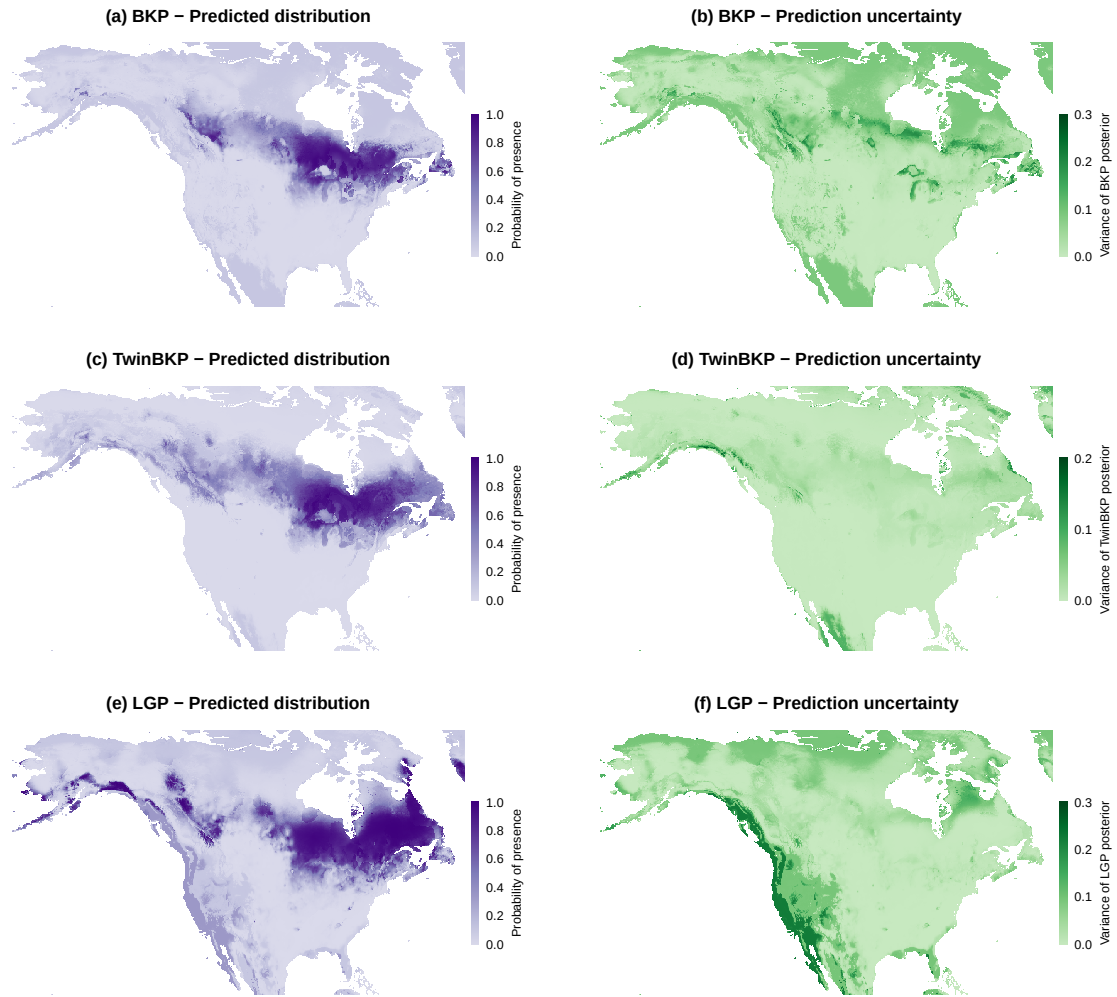


Figure 18: Spatial projections from the BKP, TwinBKP, and LGP models for the Mourning Warbler. The left column shows predicted probabilities of presence on a common scale from 0 to 1. The right column shows model-specific posterior variance summaries, with separate color scales for the three models. The top, middle, and bottom rows correspond to BKP, TwinBKP, and LGP, respectively.

Figure 18 compares the projected probability and variance surfaces. The probability panels use a common scale from 0 to 1, whereas the variance panels use model-specific scales.

All three models produce geographically structured occurrence-probability surfaces and identify broadly overlapping regions of elevated predicted presence. The fitted maps nevertheless differ in the spatial extent and fine-scale variation of these regions, reflecting differences between the full probability-scale kernel update, the TwinBKP global-local approximation, and the latent-GP construction.

The variance maps also show substantial spatial heterogeneity within each fitted model. Because the three methods use different posterior constructions and the variance panels have model-specific scales, these surfaces are most appropriately interpreted as within-model un-

certainty summaries rather than as a direct ranking of uncertainty calibration across methods. Predictions for raster cells containing environmental combinations that are weakly represented by the training data should additionally be interpreted as covariate-space extrapolations.

6. Summary and discussion

This article has presented **BKP**, a user-friendly R package for Beta Kernel Process (BKP) modeling. The package implements a Bayesian-inspired, probability-scale kernel smoothing framework for input-dependent binomial probabilities, based on kernel-weighted pseudo-count aggregation and beta-binomial conjugate updating. It provides a flexible and modular interface for fitting BKP models to both binary and aggregated binomial data, extends the same kernel-smoothed conjugate updating strategy to the Dirichlet Kernel Process (DKP) for categorical and multinomial responses, and includes TwinBKP and TwinDKP variants for twinning-based global-local approximation. To our knowledge, **BKP** is the first publicly available software dedicated to BKP and DKP methodology, thereby filling an important gap in the toolkit for modeling input-dependent binomial and multinomial probability surfaces.

The framework is intended to complement, rather than replace, latent Gaussian-process classifiers and other nonparametric probabilistic models. Its main advantages are that inference is performed directly on the probability scale, posterior summaries are available in closed form under the fitted local conjugate update, and the resulting workflow is transparent and easy to implement for binomial and multinomial response data. At the same time, BKP should be interpreted as a Bayesian-inspired local-likelihood smoothing framework, not as a fully specified Bayesian stochastic-process model in the Gaussian-process sense. Consequently, its credible intervals are pointwise posterior-style summaries under the fitted kernel-weighted update and do not carry frequentist coverage guarantees. Appendix A provides a simulation-based assessment of their empirical coverage and illustrates the effect of Shepard ESS calibration. Developing conformalized BKP procedures with finite-sample marginal coverage guarantees for future responses, together with calibrated pointwise intervals or simultaneous confidence bands for the latent probability surface, remains an important direction for future research. In addition, the full BKP and DKP implementations remain quadratic full-kernel methods in memory, whereas TwinBKP and TwinDKP provide scalable global-local approximations for larger datasets.

Future development directions include extending the BKP framework to support more complex data structures, such as multivariate responses, functional data, time series, and combinations of qualitative and quantitative covariates. Another promising avenue is to generalize the probability-scale kernel updating idea to likelihoods beyond the binomial and multinomial families. For example, negative-binomial-type likelihoods may be useful for over-dispersed count data, where the variance exceeds the mean, a common feature in ecological surveys, RNA-seq gene expression counts, and epidemiological incidence data. Geometric likelihoods, as special cases of negative-binomial models, may also be useful for modeling waiting times or the number of trials until the first success in reliability, survival, and event-process applications. Developing such extensions would require likelihood-specific conjugate or approximate updating schemes, together with appropriate uncertainty calibration and model-selection criteria.

Finally, we welcome contributions from the community and invite developers to participate in

the ongoing maintenance and extension of the package by submitting issues or pull requests through the **BKP** [GitHub repository](#).

Computational details and reproducibility

All analyses were conducted using R version 4.6.1 and **BKP** version 0.3.1. The LGP comparisons were implemented using **gplite** version 0.13.0 and **kernlab** version 0.9-33, while the Two Spirals benchmark and the **loaloe** dataset were obtained from **mlbench** version 2.1-8 and **RiskMap** version 1.0.0, respectively. R and the R packages used in this article are available from the Comprehensive R Archive Network (CRAN) at <https://CRAN.R-project.org/>. The TwinBKP and TwinDKP implementations use the header-only **nanoflann** library for kd-tree nearest-neighbour search and adapt the twinning subset-selection code from the **twingp** package. Copyright and license information for these third-party components is provided in **inst/COPYRIGHTS** of the **BKP** package. The multi-start length-scale tuning procedure follows the log-scale parameterization and search-region construction used in **GPfit**. All timing experiments reported in Figure 4 and Table 3 were conducted on a machine equipped with an Apple M4 chip (10-core CPU) and 16 GB of unified memory. GPU acceleration was not used, and all methods were run using their default single-threaded settings. The computation times in Figure 4 are averages over 20 independent repetitions, whereas those in Table 3 are reported from a single run.

Complete reproducibility materials for this article are available from the **BKP-paper reproducibility repository**. The repository contains the analysis scripts, data-processing code, generated figures, and numerical results for the illustrative examples, real-data applications, and appendix simulation. The computational environment is recorded in **renv.lock** and can be restored using **renv::restore()**. The main analysis workflow can be run from the repository root using **source("code/run_all.R")**; instructions for rerunning the computationally intensive timing experiments are provided in the repository README.

Acknowledgments

The authors thank the editorial team and the anonymous reviewers for their constructive comments and suggestions, which helped improve the clarity and reproducibility of the article.

References

- Benest J, Rhodes S, Evans TG, White RG (2022). “The Correlated Beta Dose Optimisation Approach: Optimal Vaccine Dosing Using Mathematical Modelling and Adaptive Trial Design.” *Vaccines*, **10**(11), 1838. doi:10.3390/vaccines10111838.
- Blanco JL, Rai PK (2014). “nanoflann: a C++ header-only fork of FLANN, a library for Nearest Neighbor (NN) with KD-trees.” <https://github.com/jlblancoc/nanoflann>.
- Chalup SK, Wiklendt L (2007). “Variations of the two-spiral task.” *Connection Science*, **19**(2), 183–199. doi:10.1080/09540090701398017.

- Diggle PJ, Giorgi E (2016). “Model-Based Geostatistics for Prevalence Mapping in Low-Resource Settings.” *Journal of the American Statistical Association*, **111**(515), 1096–1120. doi:[10.1080/01621459.2015.1123158](https://doi.org/10.1080/01621459.2015.1123158).
- Diggle PJ, Thomson MC, Christensen OF, Rowlingson B, Obsomer V, Gardon J, Wanji S, Takougang I, Enyong P, Kamgno J, Remme JH, Boussinesq M, Molyneux DH (2007). “Spatial modelling and the prediction of Loa loa risk: decision making under uncertainty.” *Annals of Tropical Medicine & Parasitology*, **101**(6), 499–509. doi:[10.1179/136485913X13789813917463](https://doi.org/10.1179/136485913X13789813917463).
- Dimitriadis T, Dümbgen L, Henzi A, Puke M, Ziegel J (2022). “Honest calibration assessment for binary outcome predictions.” *Biometrika*, **110**(3), 663–680. doi:[10.1093/biomet/asac068](https://doi.org/10.1093/biomet/asac068).
- Eddelbuettel D, François R (2011). “Rcpp: Seamless R and C++ Integration.” *Journal of Statistical Software*, **40**(8), 1–18. doi:[10.18637/jss.v040.i08](https://doi.org/10.18637/jss.v040.i08).
- Eddelbuettel D, Sanderson C (2014). “RcppArmadillo: Accelerating R with high-performance C++ linear algebra.” *Computational Statistics and Data Analysis*, **71**, 1054–1063. doi:[10.1016/j.csda.2013.02.005](https://doi.org/10.1016/j.csda.2013.02.005).
- Fan J, Farmen M, Gijbels I (1998). “Local Maximum Likelihood Estimation and Inference.” *Journal of the Royal Statistical Society Series B: Statistical Methodology*, **60**(3), 591–608. doi:[10.1111/1467-9868.00142](https://doi.org/10.1111/1467-9868.00142).
- Fan J, Gijbels I (1996). *Local Polynomial Modelling and Its Applications*, volume 66 of *Monographs on Statistics and Applied Probability*. 1st edition. Routledge, Boca Raton, Florida. doi:[10.1201/9780203748725](https://doi.org/10.1201/9780203748725).
- Fasshauer GE (2007). *Meshfree Approximation Methods with MATLAB*, volume 6 of *Interdisciplinary Mathematical Sciences*. World Scientific, New Jersey, NJ. ISBN 978-981-270-634-8.
- Flores G, Smith AH, Schiff AE, Fukuyama J, Wilson AC (2026). “A Consequentialist Critique of Binary Classification Evaluation: Theory, Practice, and Tools.” In *The 29th International Conference on Artificial Intelligence and Statistics*. URL <https://openreview.net/forum?id=0tOKBsnxWi>.
- Gardner J, Pleiss G, Weinberger KQ, Bindel D, Wilson AG (2018). “GPyTorch: Blackbox Matrix-Matrix Gaussian Process Inference with GPU Acceleration.” In S Bengio, H Wallach, H Larochelle, K Grauman, N Cesa-Bianchi, R Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/27e8e17134dd7083b050476733207ea1-Paper.pdf.
- Garnett R (2023). *Bayesian Optimization*. Cambridge University Press, Cambridge, United Kingdom. doi:[10.1017/9781108348973](https://doi.org/10.1017/9781108348973). <https://bayesoptbook.com/>.
- Giorgi E (2025). *RiskMap: Geostatistical Modeling of Spatially Referenced Data*. R package version 1.0.0, URL <https://CRAN.R-project.org/package=RiskMap>.

- Gneiting T, Raftery AE (2007). “Strictly Proper Scoring Rules, Prediction, and Estimation.” *Journal of the American Statistical Association*, **102**(477), 359–378. doi: [10.1198/016214506000001437](https://doi.org/10.1198/016214506000001437).
- Goetschalckx R, Poupart P, Hoey J (2011). “Continuous Correlated Beta Processes.” In *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence - Volume Two*, IJCAI’11, p. 1269–1274. AAAI Press. URL <https://www.ijcai.org/Proceedings/11/Papers/215.pdf>.
- Golding N, Purse BV (2016). “Fast and flexible Bayesian species distribution modelling using Gaussian processes.” *Methods in Ecology and Evolution*, **7**, 598–608. doi: [10.1111/2041-210X.12523](https://doi.org/10.1111/2041-210X.12523).
- GPy (since 2012). “GPy: A Gaussian process framework in python.” <http://github.com/SheffieldML/GPy>.
- Gramacy RB, Taddy M (2010). “Categorical Inputs, Sensitivity Analysis, Optimization and Importance Tempering with tgp Version 2, an R Package for Treed Gaussian Process Models.” *Journal of Statistical Software*, **33**(6), 1–48. URL <https://www.jstatsoft.org/v33/i06/>.
- Harris DJ (2015). “Generating realistic assemblages with a joint species distribution model.” *Methods in Ecology and Evolution*, **6**, 465–473. doi: [10.1111/2041-210X.12332](https://doi.org/10.1111/2041-210X.12332).
- Hastie T, Tibshirani R, Friedman J (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, volume 2. Springer New York, New York, NY. doi: [10.1007/978-0-387-84858-7](https://doi.org/10.1007/978-0-387-84858-7).
- Hjort NL (1990). “Nonparametric Bayes Estimators Based on Beta Processes in Models for Life History Data.” *The Annals of Statistics*, **18**(3), 1259–1294. doi: [10.1214/aos/1176347749](https://doi.org/10.1214/aos/1176347749).
- Johnson SG (2007). “The NLOpt nonlinear-optimization package.” <https://github.com/stevengj/nlopt>.
- Karatzoglou A, Smola A, Hornik K, Zeileis A (2004). “kernlab – An S4 Package for Kernel Methods in R.” *Journal of Statistical Software*, **11**(9), 1–20. doi: [10.18637/jss.v011.i09](https://doi.org/10.18637/jss.v011.i09).
- Leisch F, Dimitriadou E (2024). *mlbench: Machine Learning Benchmark Problems*. R package version 2.1-6, URL <https://CRAN.R-project.org/package=mlbench>.
- Loeppky JL, Sacks J, Welch WJ (2009). “Choosing the Sample Size of a Computer Experiment: A Practical Guide.” *Technometrics*, **51**(4), 366–376. doi: [10.1198/TECH.2009.08040](https://doi.org/10.1198/TECH.2009.08040).
- MacDonald B, Ranjan P, Chipman H (2015). “GPfit: An R Package for Fitting a Gaussian Process Model to Deterministic Simulator Outputs.” *Journal of Statistical Software*, **64**(12), 1–23. doi: [10.18637/jss.v064.i12](https://doi.org/10.18637/jss.v064.i12).
- MacKenzie CA, Trafalis TB, Barker K (2014). “A Bayesian Beta Kernel Model for Binary Classification and Online Learning Problems.” *Statistical Analysis and Data Mining: The ASA Data Science Journal*, **7**(6), 434–449. doi: [10.1002/sam.11241](https://doi.org/10.1002/sam.11241).

- Martin BD, Witten D, Willis AD (2020). “Modeling microbial abundances and dysbiosis with beta-binomial regression.” *The Annals of Applied Statistics*, **14**(1), 94–115. doi:[10.1214/19-AOAS1283](https://doi.org/10.1214/19-AOAS1283).
- Matthews AGdG, van der Wilk M, Nickson T, Fujii K, Boukouvalas A, León-Villagr  P, Ghahramani Z, Hensman J (2017). “GPflow: A Gaussian process library using TensorFlow.” *Journal of Machine Learning Research*, **18**(40), 1–6. URL <http://jmlr.org/papers/v18/16-537.html>.
- Montesano L, Lopes M (2009). “Learning grasping affordances from local visual descriptors.” In *2009 IEEE 8th International Conference on Development and Learning*, pp. 1–6. doi:[10.1109/DEVLRN.2009.5175529](https://doi.org/10.1109/DEVLRN.2009.5175529).
- Montesano L, Lopes M (2012). “Active learning of visual descriptors for grasping using non-parametric smoothed beta distributions.” *Robotics and Autonomous Systems*, **60**(3), 452–462. doi:[10.1016/j.robot.2011.07.013](https://doi.org/10.1016/j.robot.2011.07.013).
- Morita S, Thall PF, M ller P (2008). “Determining the Effective Sample Size of a Parametric Prior.” *Biometrics*, **64**(2), 595–602. doi:[10.1111/j.1541-0420.2007.00888.x](https://doi.org/10.1111/j.1541-0420.2007.00888.x).
- Murphy KP (2022). *Probabilistic Machine Learning: An introduction*. Adaptive Computation and Machine Learning Series. The MIT Press, Cambridge, Massachusetts. URL <http://probml.github.io/book1>.
- Mussi M, Drago S, Metelli AM (2024). “Open Problem: Tight Bounds for Bernoulli Rewards in Kernelized Multi-Armed Bandits.” In *Workshop on Aligning Reinforcement Learning Experimentalists and Theorists at the International Conference on Machine Learning (ICML)*. ArXiv: 2407.06321, URL <https://arxiv.org/abs/2407.06321>.
- Najera-Zuloaga J, Lee DJ, Arostegui I (2018). “Comparison of beta-binomial regression model approaches to analyze health-related quality of life data.” *Statistical Methods in Medical Research*, **27**(10), 2989–3009. doi:[10.1177/0962280217690413](https://doi.org/10.1177/0962280217690413).
- Najera-Zuloaga J, Lee DJ, Arostegui I (2019). “A beta-binomial mixed-effects model approach for analysing longitudinal discrete and bounded outcomes.” *Biometrical Journal*, **61**(3), 600–615. doi:[10.1002/bimj.201700251](https://doi.org/10.1002/bimj.201700251).
- Nickisch H, Rasmussen CE (2008). “Approximations for Binary Gaussian Process Classification.” *Journal of Machine Learning Research*, **9**(67), 2035–2078. URL <http://jmlr.org/papers/v9/nickisch08a.html>.
- Papadopoulos H (2024). “Guaranteed Coverage Prediction Intervals With Gaussian Process Regression.” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **46**(12), 9072–9083. doi:[10.1109/TPAMI.2024.3418214](https://doi.org/10.1109/TPAMI.2024.3418214).
- Picheny V, Wagner T, Ginsbourger D (2013). “A Benchmark of Kriging-based Infill Criteria for Noisy Optimization.” *Structural and Multidisciplinary Optimization*, **48**(3), 607–626. doi:[10.1007/s00158-013-0919-4](https://doi.org/10.1007/s00158-013-0919-4).
- Piironen J (2022). *gplite: General Purpose Gaussian Process Modelling*. doi:[10.32614/CRAN.package.gplite](https://doi.org/10.32614/CRAN.package.gplite). R package version 0.13.0, URL <https://CRAN.R-project.org/package=gplite>.

- R Core Team (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. doi:10.32614/R.manuals.
- Rasmussen CE, Williams CK (2006). *Gaussian Processes for Machine Learning*. the MIT Press, Cambridge, MA. URL <https://gaussianprocess.org/gpml/>.
- Ren L, Wang Y, Carin L, Dunson D (2011). “The Kernel Beta Process.” In J Shawe-Taylor, R Zemel, P Bartlett, F Pereira, K Weinberger (eds.), *Advances in Neural Information Processing Systems*, volume 24. Curran Associates, Inc. URL https://proceedings.neurips.cc/paper_files/paper/2011/file/04ecb1fa28506ccb6f72b12c0245ddbc-Paper.pdf.
- Rolland P, Kavis A, Immer A, Singla A, Cevher V (2019). “Efficient learning of smooth probability functions from Bernoulli tests with guarantees.” In K Chaudhuri, R Salakhutdinov (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 5459–5467. PMLR. URL <https://proceedings.mlr.press/v97/rolland19a.html>.
- Roustant O, Ginsbourger D, Deville Y (2012). “DiceKriging, DiceOptim: Two R Packages for the Analysis of Computer Experiments by Kriging-Based Metamodeling and Optimization.” *Journal of Statistical Software*, **51**(1), 1–55. doi:10.18637/jss.v051.i01.
- Rowan TH (1990). *Functional Stability Analysis of Numerical Algorithms*. Ph.D. thesis, Department of Computer Sciences, University of Texas at Austin, Austin, TX.
- Sauer J, Hines J, Fallon J, Pardieck K, Ziolkowski Jr D, Link W (2014). “The North American Breeding Bird Survey, Results and Analysis 1966–2012. Version 02.19.2014.” *Technical report*, USGS Patuxent Wildlife Research Center, Laurel, MD.
- Shepard D (1968). “A Two-Dimensional Interpolation Function for Irregularly-Spaced Data.” In *Proceedings of the 1968 23rd ACM National Conference*, pp. 517–524. doi:10.1145/800186.810616.
- Sung CL, Hung Y, Rittase W, Zhu C, Wu CFJ (2020). “Calibration for Computer Experiments With Binary Responses and Application to Cell Adhesion Study.” *Journal of the American Statistical Association*, **115**(532), 1664–1674. doi:10.1080/01621459.2019.1699419.
- Vakayil A, Joseph VR (2022). “Data Twinning.” *Statistical Analysis and Data Mining: The ASA Data Science Journal*, **15**(5), 598–610. doi:10.1002/sam.11574.
- Vakayil A, Joseph VR (2024). “A Global-Local Approximation Framework for Large-Scale Gaussian Process Modeling.” *Technometrics*, **66**(2), 295–305. doi:10.1080/00401706.2023.2296451.
- Vehtari A, Gelman A, Gabry J (2017). “Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC.” *Statistics and Computing*, **27**(5), 1413–1432. doi:10.1007/s11222-016-9696-4.
- Wen H, Betken A, Hang H (2025). “Optimal Learning of Kernel Logistic Regression for Complex Classification Scenarios.” In *The Thirteenth International Conference on Learning Representations*. URL <https://openreview.net/forum?id=WlhVRh2rQ0>.

- Wendland H (1995). “Piecewise Polynomial, Positive Definite and Compactly Supported Radial Functions of Minimal Degree.” *Advances in Computational Mathematics*, 4(1), 389–396. doi:10.1007/BF02123482.
- Zhao J, Qing K, Xu J (2026a). *BKP: Beta Kernel Process Modeling*. R package version 0.3.1, URL <https://cran.r-project.org/package=BKP>.
- Zhao J, Shi X, Xu J (2026b). “Shared Keyboard: An improved Bayesian design for phase I clinical trials via Beta kernel process.” ArXiv: 2605.25043, URL <https://arxiv.org/abs/2605.25043>.

A. Pointwise Interval-Coverage Simulation

We evaluate the empirical pointwise coverage of the uncertainty intervals produced by **BKP** using the one-dimensional oscillatory probability function from Example 2 in Equation 17. The comparison includes standard BKP with `ess = "none"`, Shepard effective-sample-size calibrated BKP with `ess = "shepard"`, hereafter denoted BKP-ESS, and a logistic Gaussian process (LGP).

For an evaluation set $\mathcal{E} = \{x_1, \dots, x_N\}$, the empirical pointwise coverage rate in one simulation replication is defined as

$$\hat{C}(\mathcal{E}) = \frac{1}{N} \sum_{i=1}^N \mathbb{1} \{L(x_i) \leq \pi(x_i) \leq U(x_i)\},$$

where $L(x_i)$ and $U(x_i)$ are the lower and upper bounds of a nominal 95% pointwise interval for the latent probability $\pi(x_i)$.

We consider training sample sizes $n = 30$ and $n = 100$, with 100 independently generated datasets for each setting. In each replication, the training inputs are generated by Latin hypercube sampling over $[-2, 2]$, the binomial trial sizes are sampled from $\{1, \dots, 100\}$, and the responses are generated from the true probability function in Equation 17. All three models, including their hyperparameter optimization procedures, are refitted independently in every replication. The BKP and BKP-ESS intervals are obtained from the quantiles of the fitted beta distributions, whereas the LGP intervals are obtained from the transformed posterior quantiles returned by **gplite**.

Coverage is evaluated on two sets of input locations. The evaluation grid consists of the same 2,000 equally spaced points over $[-2, 2]$ in every replication and is used to assess coverage throughout the input domain. The training-location coverage is evaluated at the n randomly generated training inputs from the corresponding replication. The evaluation grid contains no additional observed responses and should therefore not be interpreted as a conventional held-out testing dataset.

The nominal 95% interval level is not an automatic finite-sample frequentist coverage guarantee for either BKP or LGP. Both methods produce model-based posterior intervals whose empirical coverage may depend on the kernel specification, likelihood, prior, and hyperparameter estimation. Conventional Gaussian-process intervals may similarly exhibit substantial under-coverage under model misspecification (Papadopoulos 2024).

Figure 19 shows representative fitted probability surfaces and pointwise 95% intervals from one simulation replication. Table 4 summarizes the mean and standard deviation of the replication-specific coverage rates across the 100 replications.

The standard BKP and LGP intervals exhibit under-coverage in all four settings. BKP-ESS produces coverage close to the nominal level for $n = 30$, with mean coverage of 94.1% on the evaluation grid and 94.6% at the training locations. For $n = 100$, its mean coverage increases to 97.6% for both evaluation sets, indicating moderately conservative intervals. The standard deviations are also smaller for BKP-ESS, particularly when $n = 100$.

These results show that Shepard ESS calibration substantially changes the concentration of the BKP intervals and reduces under-coverage under the present simulation design. They do not establish a general frequentist coverage guarantee, because coverage may depend on the data-generating mechanism and model specification.

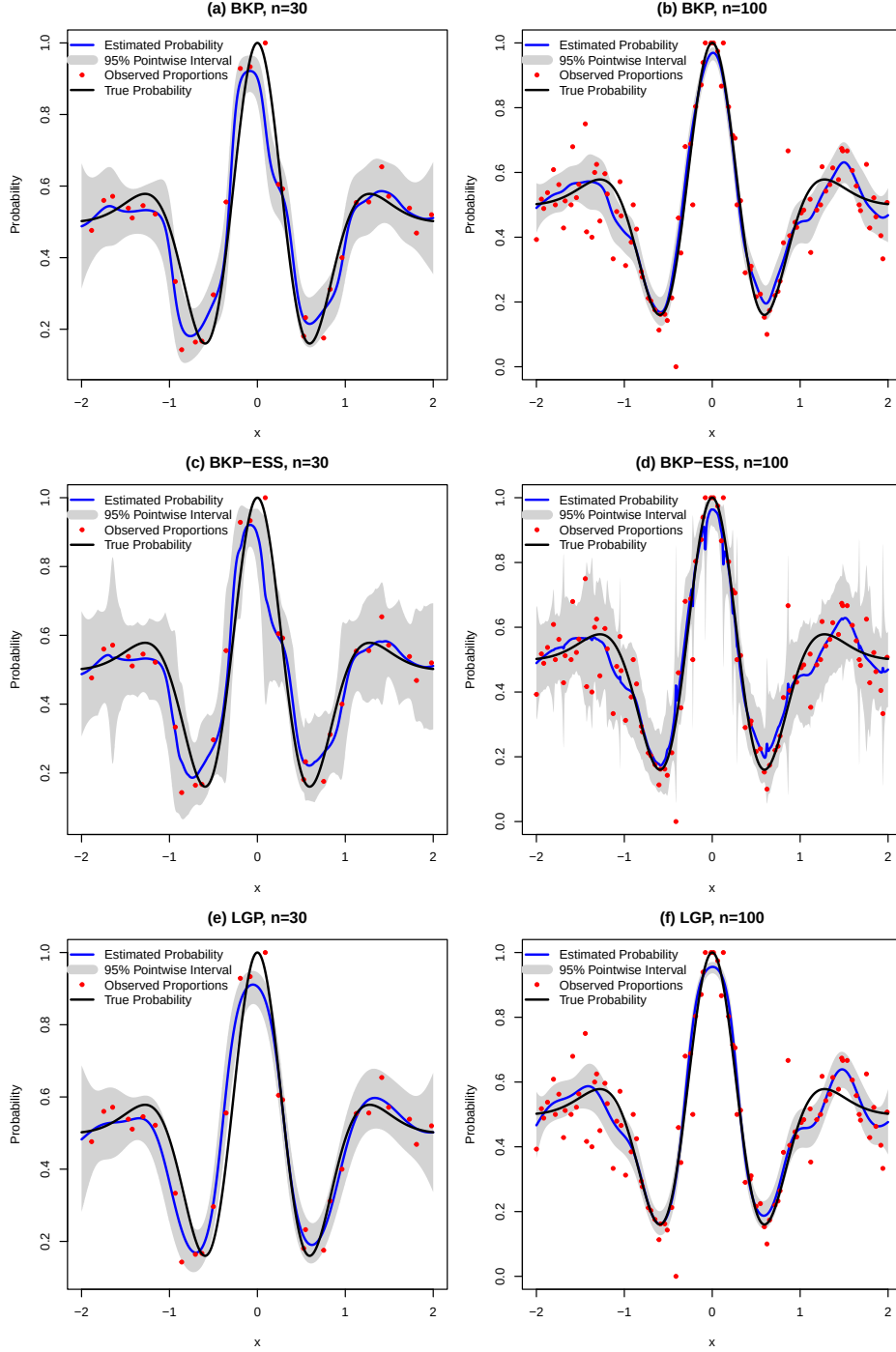


Figure 19: Representative fitted probability surfaces and pointwise 95% intervals for Example 2. The rows correspond to BKP, BKP-ESS, and LGP, respectively, while the columns correspond to training sample sizes $n = 30$ and $n = 100$. Each panel is based on one simulation replication and is evaluated on the fixed evaluation grid of 2,000 points.

A natural direction for future research is to develop conformalized BKP procedures that pro-

Evaluation set	Sample size	Model	Coverage (%)
Evaluation grid	$n = 30$	BKP	87.1 (6.4)
		BKP-ESS	94.1 (3.1)
		LGP	91.4 (4.0)
	$n = 100$	BKP	90.9 (4.1)
		BKP-ESS	97.6 (1.0)
		LGP	89.5 (3.6)
Training locations	$n = 30$	BKP	88.1 (6.7)
		BKP-ESS	94.6 (3.3)
		LGP	91.1 (5.0)
	$n = 100$	BKP	90.8 (4.1)
		BKP-ESS	97.6 (1.0)
		LGP	89.8 (3.7)

Table 4: Empirical coverage of nominal 95% pointwise intervals. Entries are mean coverage percentages across 100 independent replications, with standard deviations of the replication-specific coverage rates in parentheses. The evaluation-grid results use the same 2,000 equally spaced points over $[-2, 2]$ in every replication, whereas the training-location results use the randomly generated training inputs from each replication.

vide finite-sample marginal coverage guarantees for future Bernoulli, binomial, or multinomial responses under exchangeability. BKP posterior summaries or beta-binomial and Dirichlet-multinomial predictive distributions could be used to construct suitable nonconformity scores. Coverage guarantees for the latent probability surface $\pi(x)$ are distinct from response-level conformal coverage and require separate development of calibrated pointwise intervals or simultaneous confidence bands.

Affiliation:

Jiangyan Zhao, Kunhai Qing (*equal contribution*)
School of Statistics
East China Normal University

Jin Xu (*corresponding author*)

School of Statistics

and

Key Laboratory of Advanced Theory and Application in Statistics and Data Science - MOE

East China Normal University

3663 North Zhongshan Road

Shanghai 200062, China

E-mail: jxu@stat.ecnu.edu.cn

URL: https://faculty.ecnu.edu.cn/_s35/xj2_en/main.psp

Journal of Statistical Software

published by the Foundation for Open Access Statistics

MMMMMM YYYY, Volume VV, Issue II

[doi:10.18637/jss.v000.i00](https://doi.org/10.18637/jss.v000.i00)

<http://www.jstatsoft.org/>

<http://www.foastat.org/>

Submitted: yyyy-mm-dd

Accepted: yyyy-mm-dd
